EUCLIDEAN GEOMETRY AND AI

STEVEN CREECH AND PETER ZENZ

Abstract. The following document consists of lecture notes that came out of a special topics course at Brown University in Fall of 2025 on Euclidean Geometry and Artificial Intelligence. The hope is that this document is created in such a way that others can replicate this course at their own universities. In addition to this document, there is an accompanying github repository where there are coded outlines for homeworks with corresponding tests for the code.

Contents

Date: June 28, 2026.

Completion of the writing:

(1)
(2)
(3) Arichitecture of Our System
(4)
(5) Basic Geometric Objects
(6) Basic Predicates
(7) Problem Data Structure
(8) Deductive Database
(9) Generating Conjectures
(10) Algebraic Reasoning
(11) Implementing Area
(12) Auxliary Constructions
(13) Generating Synthetic Data
(14) Cleaning the Data
(15)
(16)
(17)
(18)
(19)
(20)
(21)
(22)
(23)
(24)
(25)
(26)
(27) Homework

1. Introduction

There are two aspects to this course: learning geometry in a broad sense and building an AI that can solve Euclidean geometry problems. In terms of geometry, the topics covered include: powers of points, radical axis, Ceva's theorem, Menelaus's theorem, Pappuss's Hexagon theorem, Pascal's theorem, and Brianchon's theorem. After talking about these more classical results of Euclidean geometry. The course then turned to Hilbert's axiomization of geometry [Har13].

As for the coding aspect of the course, in building the AI the topics covered include: formalizing geometry problems, neuro-symbolic reasoning, generating random diagrams, creating synthetic data, cleaning data, n-shot prompting and fine-tuning language models. We also include a few references to automatic diagram generation and auto-formalization, but we do not go into these topics, the ambitious students may attempt to incorporate these ideas into their systems.

By the end of the course, students will have created an AI that is akin to that of Google Deepminds AlphaGeometry [CTO+25].

The course was structured in a way that there are 3 lectures in a week. 2 lectures are about geometry, and then there is a lecture about implementing and building the AI. These two concepts go hand in hand, but the main focus of these notes are on the implementation. At the beginning of the semester some of the mathematical lectures were used to discuss formal language, and conventions used. This document is more concerned with the implementation of the AI; thus, the beginning focus on the mathematics is more concerned with setting up a uniform notation for the AI systems. At the end of these notes, we also include several of the more classical theorems of Euclidean geometry discussed in the course with some discussion on caveats if these were to be implemented into our systems.

During the original offering of the course, we left the coding aspect open ended. This was good in that some students came up with some great ideas in their implementations; however, it also led to students having several issues with bugs in their work. Thus, in this document, we give a more structured outline of the coding that comes from some of the ideas of the students. However, we encourage any students following through this document and building their own system if they have any optimizations or additions they should add them. By doing this, we hope that in future courses, students will have a combination of both structure and freedom in creating their AI.

A Statement on AI Usage: In the initial run of this course, we encouraged students to use AI to aid them in the creation of their systems. Furthermore, in the creation of these notes, AI was used in helping of the creation of diagrams and synthesizing the prose, and creating code outlines and tests for homework. Everything that was generated by an AI was looked over by a human, and we expect that students to also go over all artifacts created by AI.

2. AlphaGeometry Overview

The following notes contains a high level overview of how the AlphaGeometry system works.

What is AlphaGeometry: It is a neuro-symbolic reasoning system to solve Euclidean geometry problems. The **symbolic reasoning** corresponds to using formal logic and rule based reasoning. This is good because by combining theorems using logical rigor will guarantee the correctness of the solution. The **neuro network** corresponds to using pattern recognition and learning from a large set of data to solve the problem. In principal, if prompting a LM with a geometry problem there is no guarantee that the solution is correct and that the LM did not hallucinate. Thus, the power of AlphaGeometry comes in combining the two strategies. We use the **neuro** aspect to come up with ideas, and then we use the **symbolic** aspect to give precise reasoning and insure that the solution is correct.

There are three main components to AlphaGeometry:

(1) DD (deductive database)
(2) AR (algebraic reasoning)
(3) LM (language model)

DD and AR correspond to the **symbolic** aspect of the system, and the LM corresponds to the **neuro** aspect. In particular, DD corresponds to a list of rules/theorems that combine different statements to deduce new statements. AR then uses linear algebra to speed up the reasoning done with DD. The general idea is to use DDAR to solve the problem; however, if the problem cannot be solved by DDAR, then the LM is prompted and asked to give an auxiliary construction. Then the hope is that with this auxiliary construction, then

DDAR can solve the problem. We shall call these auxiliary constructions **rabbits** as a magician pulls a rabbit out of a hat when we are given this magical rabbit, then the symbolic system of DDAR can solve the problem.

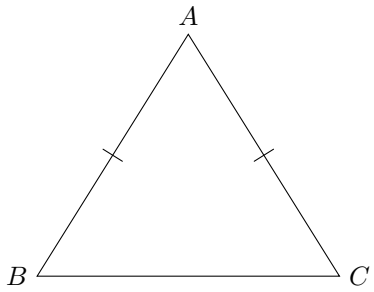
The LM knows which rabbits to give since it will be trained on theorems and proofs. Thus, by looking at patterns in proofs it has been trained on, the LM will suggest rabbits that are useful in solving the given problem.

Example 2.1. *Let ABC be any triangle with $AB = AC$. Prove that $\angle ABC = \angle ACB$.*

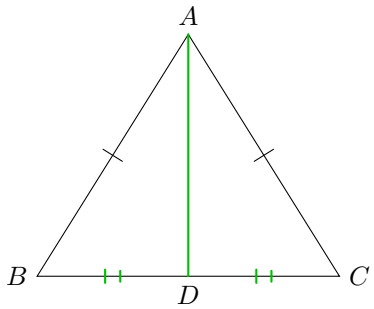
In order to prove this, we must first give the computer a formalized version of this natural language problem. Such as

- `[1] ncol A B C` *(this will say that the points A, B, C are not collinear that is that they form a triangle)*
- `[2] cong A B A C` *(this will say that the segment AB is congruent to the segment AC)*
- `[?] eqangle A B C A C B` *(this is saying that we are asking if the angle ABC is equal to the angle ACB)*

We must also supply the computer with a diagram of the problem (Google Deepmind's AlphaGeometry has an autoformalizer and auto diagram generator; however, in this course we will supply the computer with both of these).



While this problem in principle is just saying that in an isosceles triangle the two angles opposite of the two sides of the same length are the same. This could be considered as a rule in DD, in which we could just apply the rule to conclude the theorem. However, let us say that we have a smaller DD in which we only know SSS congruence of triangles. Then as written we cannot solve the problem; thus, we might prompt the LM. Say that the LM suggests the following construction: Let D be the midpoint of BC (i.e. `midp D B C`*). Then the picture becomes*



After this prompt of the LM, we can use SSS congruence to conclude the theorem.

- `[1] ncol A B C`
- `[2] cong A B A C`
- `[3] midp D B C` *(this is the **rabbit** suggested by the LM)*
- `[4] cong B D D C` *(this is implied from* `[3]`*)*
- `[5] contri A B D A C D` *(here we are saying that the triangle ABD is congruent to the triangle ACD, this follows from SSS congruence as we have that* `[2] cong A B A C`*,* `[4] cong B D D C`*, and trivially* `cong B D B D`*)*
- `[?] eqangle A B C A C B` *(we may now deduce our goal as this follows from the definition of congruent triangles* `5 contri A B D A C D`*)*

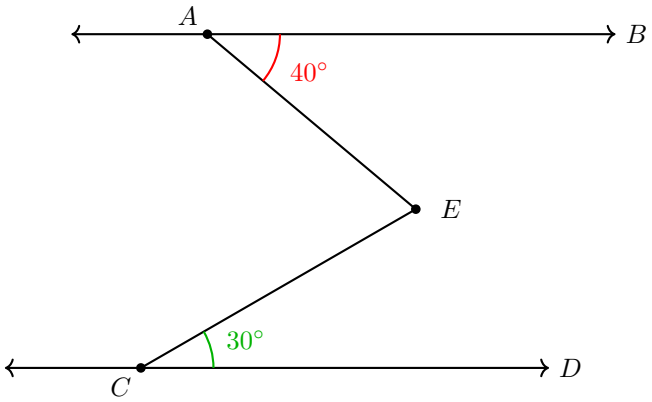
Remark 2.2. *We remark that this isn't the best example since in particular since we might remark that since*

- `[1] ncol A B C`
- `[2] cong A B A C`
- `[3] cong B C C B` *(this is a trivial statement which would either have to either be)*
- `[4] contri A B C A C B` *(here we are using the SSS congruence since we have* `cong A B C A`*,* `cong B C C B`*, and* `C A B A` *where some of these are deduced from the others by symmetries).*

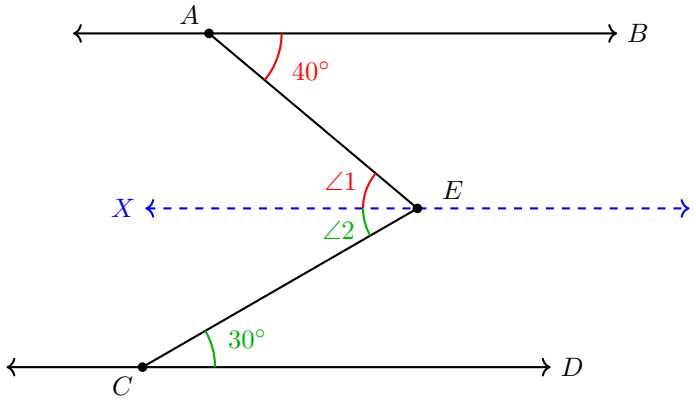
This shows that there are some subtleties in when dealing with points/triangles which involve the same set of points. Furthermore, the use of symmetries in the predicates need to be taken into account. Furthermore, for triangles, we see that the order of points matters as `contri A B C A B C` *is trivially true, but* `contri A B C A C B` *is not necessarily true. We will later discuss the orientations as a distinction between* `contri1` *and* `contri2`*.*

Example 2.3. *Here is an additional example. Say that we are given to parallel lines* `para A B C D`*, then let* `E` *be a point which lies between the two lines. Now assume that* $\angle BEC = 40°$ *and* $\angle CEA = 30°$*, then we might ask what the measurement of* $\angle AEC = ?$*. We might formalize this in the following way:*

- `[1] para A B C D`
- `[2] aconst B A E 40` *(this predicate is shorthand for angle constant which says* $\angle BAE = 40°$*)*
- `[3] aconst C E A 30`
- `[?] acompute C E A` *(this predicate is asking to compute* $\angle ACE$*)*

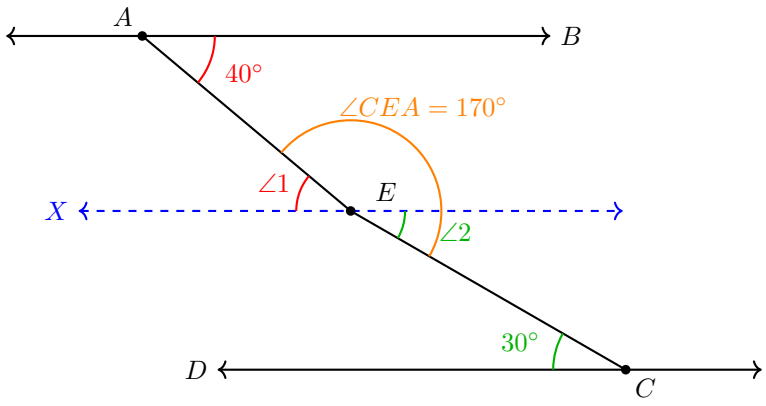


To compute this, we might consider drawing a line through E *which is parallel to both* $\overline{AB}$ *and* $\overline{CD}$*, and call this point* X*.*



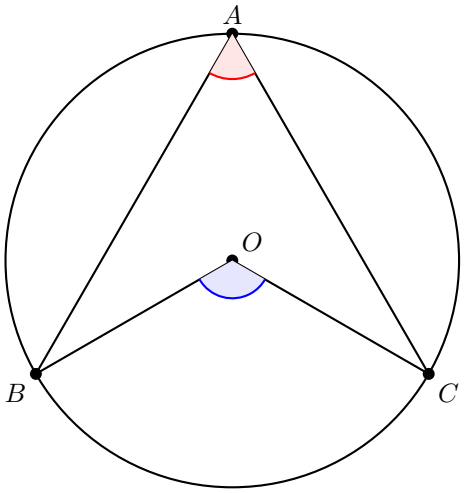
Now using alternate interior angles we will have that $\angle 1 = \angle XEA = 40°$ *and* $\angle 2 = \angle CEX = 30°$*. Thus, we see that* $\angle CEA = \angle CEX + \angle XEA = 30° + 40° = 70°$*. We would need rules that would have to formalize how we are combining angles, but in practice we will not need this rabbit as we will use linear algebra and AlphaGeometries AR system will be able to do this computation without the need of the rabbit of drawing the line through* X*.*

Remark 2.4. *Again, this example shows another subtlety. Namely, the way in which we draw the diagram matters. We have drawn the diagram so that AB and CD are read left to right in alphabetical order. However, let us consider the same problem, but let us draw the diagram in a way in which the labels of C and D are in the opposite order.*



We now see that the proof in which we gave is not true for the diagram above. This shows the subtlety which must be taken into account when dealing with angles. In particular, we will discuss different angle conventions which we denote $\angle_0$ and $\angle_1$. We will discuss how $\angle_1$ which is the angle between two lines modulo π (or $180°$) will be the one in which AlphaGeometry and our systems use.

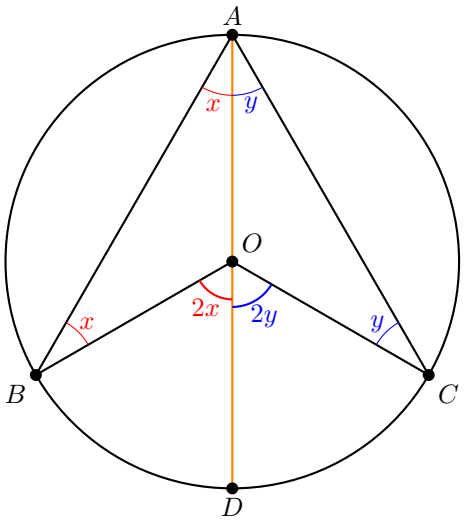
Example 2.5. *Now let us consider the following example, consider a circle with center O and ABC lying on the circle. We wish to show that $\angle BOC = 2\angle BAC$.*



We formalize this problem as follows:

- `[1] circle O A B C` *(this says that O is the circumcenter of triangle ABC)*
- `[?] angeq C A A B C O O B 2 -2 -1 1 0` *(this is asking if $2d(CA) - 2d(AB) - d(CO) + d(OB) = 0$ we will define the function d later when we talk about our angle conventions. In particular, we will have that $d(CA) - d(AB) = \angle CAB$ Thus, this is asking if $\angle BOC = 2\angle BAC$)*

Now to solve this problem, we can consider the rabbit of drawing the line from A to O intersecting the circle at a new point D.



Now once we draw this line, we observe that we will have isoceles triangles ABO and ACO with angles x and y respectively. We then have $\angle BOA = 180° - 2x$, so $\angle BOD = 2x$. Similarly, $\angle COA = 180° - 2y$, so $\angle COD = 2y$. Hence, $\angle BOC = 2x + 2y$. Similarly, we see that $\angle BAO = x$, $\angle CAO = y$, so $\angle BAC = \angle BAO + \angle CAO = x + y$. Thus, we have solved the problem.

3. Architecture of Our System

In this section, we will give a giant overview of all the files which we shall be creating through the course of the semester, and their purpose.

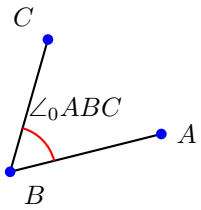
WRITE THIS ONCE TO BLUEPRINT EVERYTHING

4. Angle Conventions

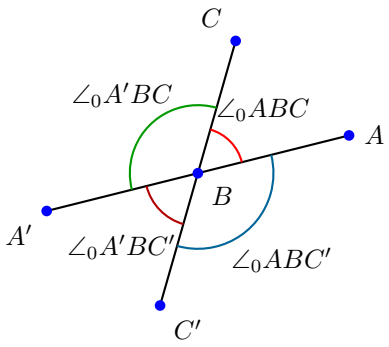
We begin by talking about the different angle conventions that one may take, and what we truly mean by an angle in terms of our system. In practice, we will consider the angle measured between two lines counter-clockwise modulo π.

4.1. **Angle 0.**

Definition 4.1. *We define $\angle_0 ABC$ to be the traditional angle measured between the segment AB and the segment BC.*



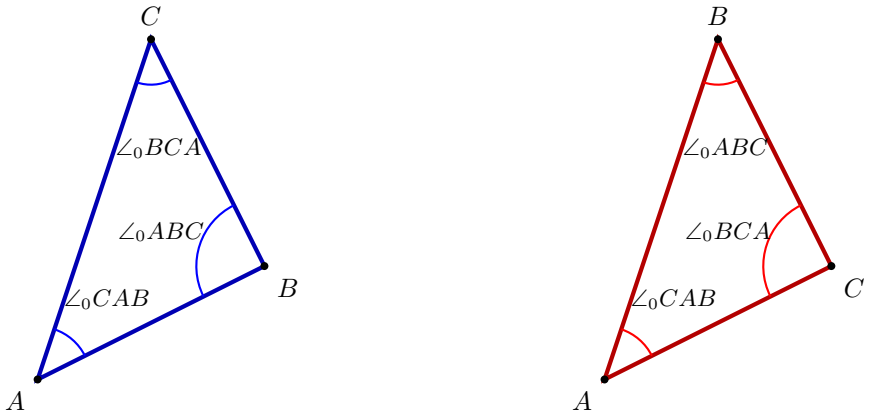
One limitation of this definition is that when we want to calculate the angle between two lines, it the measurement of the angle depends upon which side of the line the point lies on. In particular, we see that if A and A' lie on opposite sides of the line, then $\angle_0 ABC \neq \angle_0 A'BC$ (although, we will have that $\angle_0 ABC + \angle_0 A'BC = \pi$, see the figure below). Thus, if we wanted to use the convention of $\angle_0$, then for a given line, we would need to keep track of the order of the points appearing (this is possible, but more complicated to implement for the computer).



Now we see that the range of $\angle_0$ is $(0, \pi)$. We also have that $\angle_0$ satisfies $\angle_0 ABC = \angle_0 CBA$. Now in terms of $\angle_0$, the statement that the sum of the angles of a triangle add up to π can be interpreted as the following proposition.

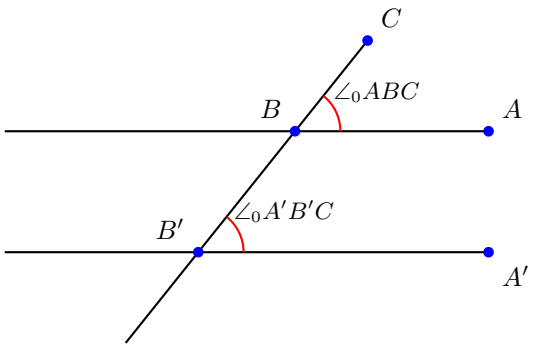
Proposition 4.2. *Given a triangle ABC, then $\angle_0 ABC + \angle_0 BCA + \angle_0 CAB = \pi$.*

This is seen by drawing the picture of the triangle, and just labeling the angles. Furthermore, we remark that this statement is independent of the orientation of the points ABC as $\angle_0 ABC = \angle_0 CBA$. In the below, the blue triangle on the left shows this in the case where the ordered points ABC are positively oriented, and the red triangle on the right shows this in the case where the ordered points ABC are negatively oriented.

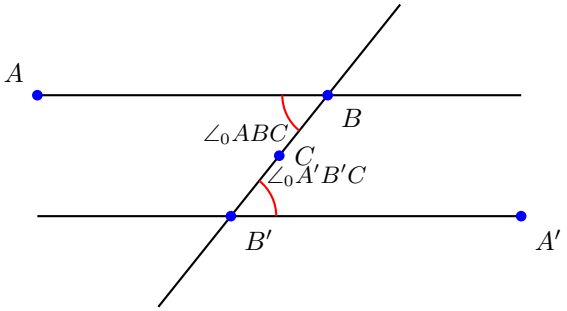


Now we remark that given parallel lines, if a third line passes through the two, then there are some statements on how some angles will be the equal. We state these rules below in terms of $\angle_0$. It is worth noting that these statements can get quite complicated where we must describe the relationship of different points to one another. This is one down side of the $\angle_0$ in that for a human it can be clear when two angles are equal, but for a computer there would be a lot of things to check to apply these angle congruences.

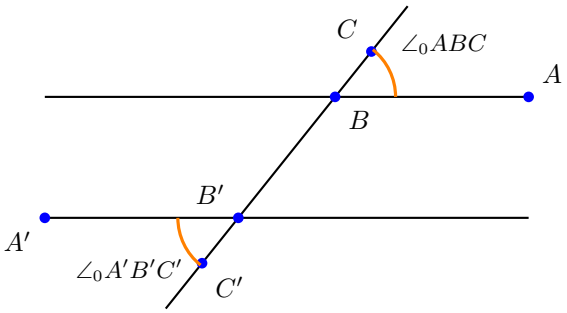
Proposition 4.3 (Corresponding Angles)**.** *Say that the lines AB and $A'B'$ are parallel and say that $BB'C$ are collinear such that A and A' lie on the same side of the plane with respect to the line $BB'C$, and C does not lie between B and B', then $\angle_0 ABC = \angle_0 A'B'C$.*
(Note: the picture below is the case where B lies between B' and C, but you may convince yourself that the statement is still true if instead B' is between B and C (that is if C is on the other end of the line)



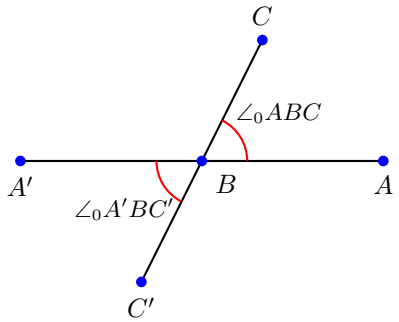
Proposition 4.4 (Alternate Interior Angles)**.** *Say that the line AB and $A'B'$ are parallel and say that BCB' are collinear such that A and A' lie on the opposite sides of the plane with respect to the line BCB', and if C lies between B and B', then $\angle_0 ABC = \angle_0 A'B'C$.*



Proposition 4.5 (Alternate Exterior Angles)**.** *Say that the lines AB and $A'B'$ are parallel and say that $C'B'BC$ are collinear such that A and A' lie on opposite sides of plane with respect to the line $C'B'BC$. Furthermore, suppose that B' is between C' and B, and that B is between C and B', then $\angle_0 ABC = \angle_0 A'B'C'$.*



Proposition 4.6 (Vertical Angles)**.** *Given lines $A'BA$ and $C'BC$ such that B lies between A and A' on the line ABA' and B lies between C and C' on the line CBC' (equivalently if A and A' are on opposite sides of the plane with respect to the line CBC' and similar with C and C' with the line ABA'), then $\angle_0 ABC = \angle_0 A'BC'$.*



Definition 4.7. *Two triangles $\triangle ABC$ and $\triangle DEF$ are congruent if $AB = DE$, $BC = EF$, $CA = FD$, $\angle_0 ABC = \angle_0 DEF$, $\angle_0 BCA = \angle_0 EFD$, and $\angle_0 CAB = \angle_0 FDE$.*

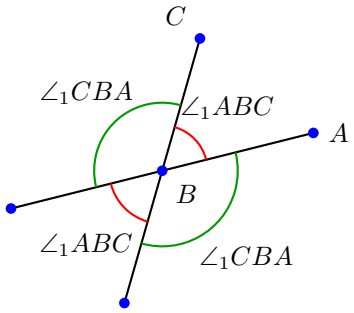
We have some classical results involving triangles.

Proposition 4.8 (Euclid Book I Prop 4)**.** *If $AB = DE$, $BC = EF$, $\angle_0 ABC = \angle_0 DEF$, then $\triangle ABC$ and $\triangle DEF$ are congruent. This is the classical "side-angle-side" triangle congruence rule.*

Proposition 4.9 (Euclid Book I Prop 5)**.** *If $AB = AC$, then $\angle_0 ABC = \angle_0 ACB$.*

4.2. **Angle 1.** Now we will describe a different way of measuring angles. This will be the measurement that we will use when building our AI. In particular, it will be important to see that there are differences in the statements of the propositions between the different kinds of angle conventions.

Definition 4.10. *We define $\angle_1 ABC$ to be the angle measured between line AB and line BC where we measure the angle counter-clockwise from the line AB to the line BC.*



We see that $\angle_1$ will take values of angles between 0 and π or 0 and 180°. We shall call this the unsigned angle modulo π or the unsigned angle modulo 180°.

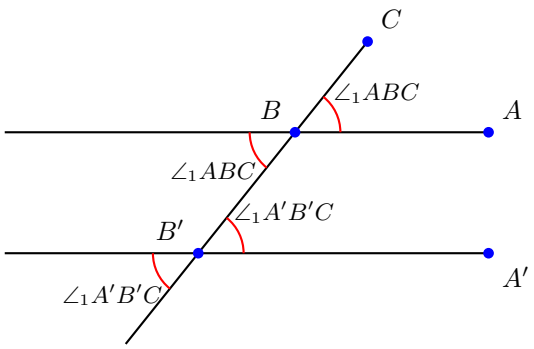
Remark 4.11. *We say it is modulo π or 180° since after rotating the line, any additional rotation by any multiple of π or 180° will be the same. Later on we will see that this modulo π condition is needed when we have that $\angle_1 ABC = d(AB) - d(BC)$ which is only true if we consider things moduluo π.*

The big advantage of defining $\angle_1$ is that this definition of angle just measures the angle between two lines. Now we see that in the definition of $\angle_1$, it is independent of where the point is on the line. However, it depends on the order of the lines. We see that $\angle_1 ABC + \angle_1 CBA = \pi$. Thus, in general $\angle_1 ABC \neq \angle_1 CBA$.

The first advantage that we see when looking at $\angle_1$ is that the statement of the vertical angles is now clearly true as we are just measuring the angle between two lines, we will have that the statement of vertical angles is just $\angle_1 ABC = \angle_1 ABC$, so we do not need to include this proposition.

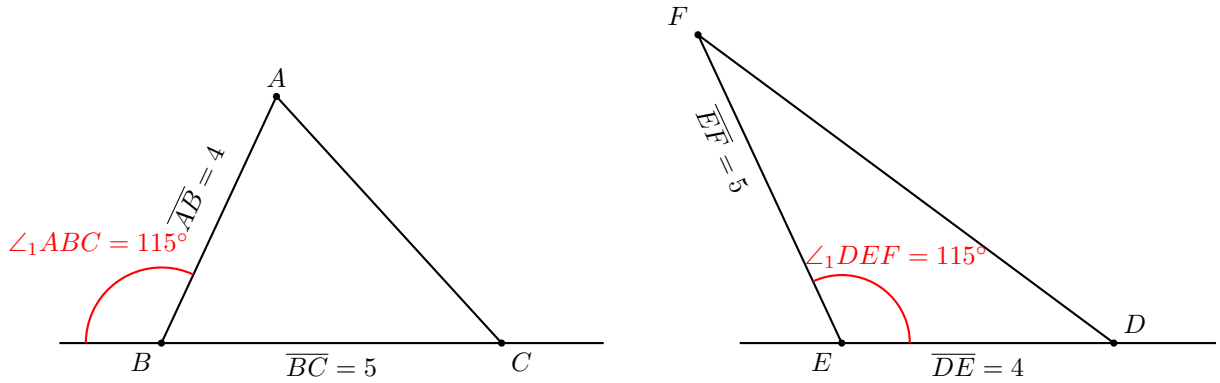
The next advantage of $\angle_1$, when we look at parallel lines then the Corresponding Angles, Alternate Interior Angles, and Alternate Exterior Angles propositions can be combined into a single statement.

Proposition 4.12 (Corresponding Angles, Alternate Interior Angles, and Alternate Exterior Angles.)**.** *Let lines AB and $A'B'$ be parallel and let $BB'C$ be collinear (where the location of C need not matter). Then $\angle_1 ABC = \angle_1 A'B'C$.*



While using $\angle_1$ gives more flexibility, one must be careful with statements of theorems. We see this as follows.

Example 4.13. *Consider Euclid's Book I Prop. 4, which is the classical side-angle-side triangle congruence rule. Let's consider the statement with $\angle_0$: If $AB = DE$, $BC = EF$, $\angle_0 ABC = \angle_0 DEF$, then $\triangle ABC$ and $\triangle DEF$ are congruent. However, we see that if we attempt to replace $\angle_0$ with $\angle_1$, then the statement is not true. Let us see this, by observing the following two triangles where $AB = DE = 4$, $BC = EF = 5$, and $\angle_1 ABC = \angle_1 DEF = 115°$.*

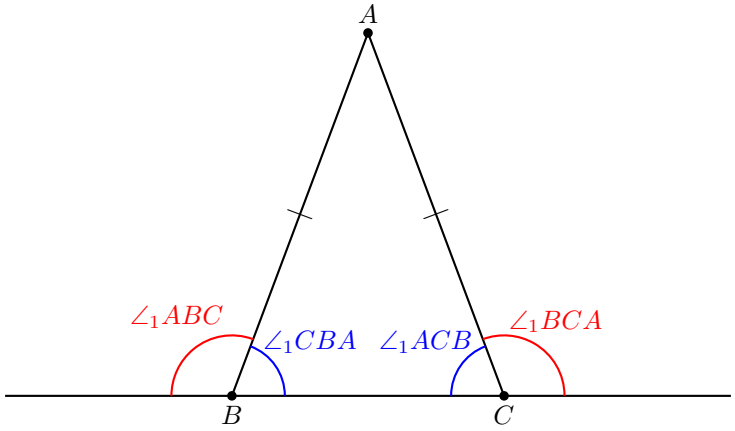


However, just by looking at these triangles it is clear that they are not congruent. The key difference between the two diagrams is the orientation of the points. We see that ABC are oriented counter-clockwise while the points DEF are oriented clock-wise. Thus, we must add a predicate `sameclock A B C D E F` *in order to track the orientations of points (in practice we can just read off the diagram the orientation of points).*

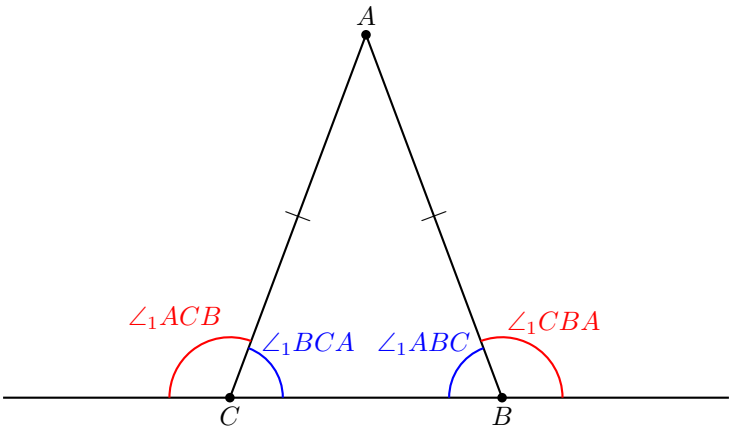
Proposition 4.14 (Euclid Book I Prop 4)**.** *If $AB = DE$, $BC = EF$, $\angle_1 ABC = \angle_1 DEF$, and the orientation of the points ABC are the same as that of DEF, then $\triangle ABC$ and $\triangle DEF$ are congruent.*

Thus, it seems that it might be the case where the `sameclock` proposition is going to be the solution to any $\angle_1$ statements; however, we shall see in the next example that adding the `sameclock` won't always be the fix to propositions.

Example 4.15. *Consider Euclid Book I Prop 5, which is says in the $\angle_0$ case that: If $AB = AC$, then $\angle_0 ABC = \angle_0 ACB$. We might ask if it is the case, that this is still true with $\angle_1$ if we replace $\angle_0$ with $\angle_1$. However, it is clear that* `sameclock A B C A B C`*. However, it is the case that $\angle_1 ABC \neq \angle_1 ACB$. We see this by drawing the diagram.*



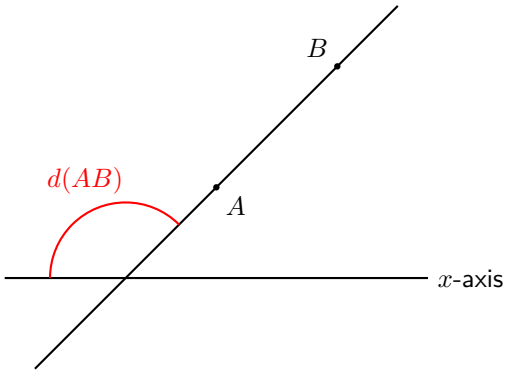
Thus, we see that $\angle_1 ABC \neq \angle_1 ACB$, but we have that $\angle_1 ABC = \angle_1 BCA$ and $\angle_1 ACB$ and $\angle_1 CBA$. Furthermore, we can see that this is independent of the orientation of the points by drawing the other orientation of the picture.



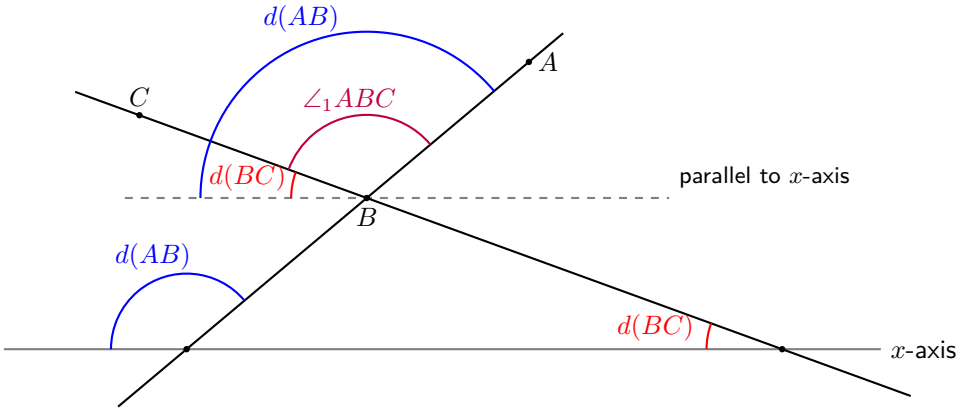
Proposition 4.16 (Euclid Book I Prop 5)**.** *If $AB = AC$, then $\angle_1 ABC = \angle_1 BCA$. (We remark that $\angle_1 ACB = \angle_1 CBA$ can be deduced from how $\angle_1$ works)*

4.3. **Angle between x-axis.** We shall now describe the computational short-cut that is needed, and will be one the most powerful tool of AlphaGeometry to make it run fast. In particular, we have defined $\angle_1$ between two lines. However, it will be more useful to just give an angle for a single line. To do this, we shall consider the x-axis as a reference point.

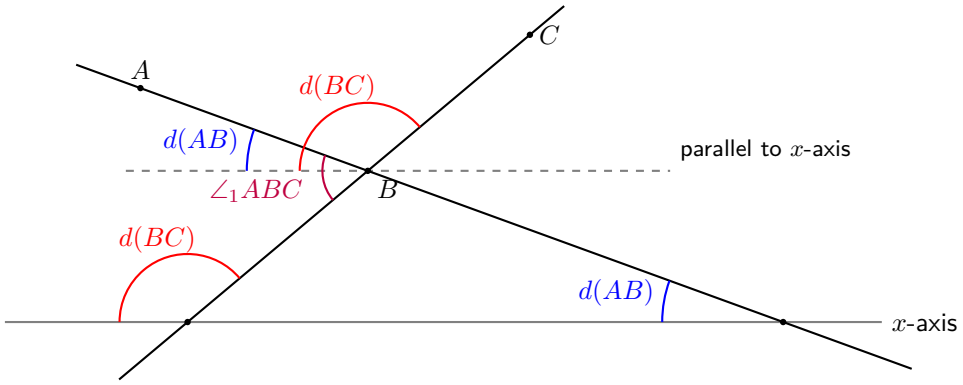
Definition 4.17. *For a line AB, we define the function $d(AB)$ to be the $\angle_1$ measure between the line AB and the x-axis.*



We will discuss the importance of $d(AB)$ as it is more versatile when implementing AR. Now we can see how to measure $\angle_1$ using d. In particular, we have that $\angle_1 ABC = d(AB) - d(BC)$.



If we consider this picture, it is clear that we get $\angle_1 ABC = d(AB) - d(BC)$; however, we must again be careful that our picture is not misleading us. Thus, let us consider the same diagram, but switch the locations of A and C.



In this picture, we now see the subtlety. In particular, we have that $d(AB) - d(BC)$ will be negative. Furthermore, the angle measurement $|d(AB) - d(BC)|$ is not $\angle_1 ABC$, so we simply cannot take absolute value. However, we see that

$$\angle_1(ABC) + d(BC) - d(AB) = 180°.$$

Thus, we will have that

$$\angle_1 ABC \equiv d(AB) - d(BC) \pmod{180°}.$$

This is why we must consider the angle modulo 180°, and have that $\angle_1 ABC$ is measured modulo 180°.

5. Basic Geometric Objects

In this section, we will discuss the basic geometric objects that we will store in the system. In practice, we could create our entire system only using **points** and building our predicates out of these points. However, in practice when solving geometry, we look at diagrams, and by looking at the diagrams, we can then reason and make conjectures. For our AI, this amounts to looking at the different objects which are present in the diagram, and then by hashing them, we can make conjectures that may be true based off our diagram. Thus, we will only try to prove things which might be true since if in our diagram there is two angles aren't equal, we shouldn't try to prove that they are and therefore waste time computing. Each of these should be made their own `class` in the `geometry.py` file.

5.1. **Points.** A **point** is the most basic geometric object. Since all predicates are built in terms of points, when implementing our system we shall build a `Point` class. These should have the following data associated to the `Point`

- `x` this will be a float that represents the x-coordinate of the point. To determine this point we read it off the diagram of the problem.
- `y` this will be a float that represents the y-coordinate of the point.
- `name` This is a string that is just the name of the point.
- `constructed` This will be a boolean that tells us if the point was constructed during the problem.
- `index` (this is just a global index for when this point was created to)
- `depth` this is a marker to denote in what order that the point was constructed. A point given in the statement of the problem should have depth 0. Points created after the first construction call should have depth 1, and so on.
- `overlaps` this is a boolean that checks to see if this point overlaps with another point

Combinatorially, we shall assume that we have n points in our problem (of course this number will grow), but we will measure all the other number of geometric objects in terms of the number of points.

5.2. **Lines.** The idea behind the lines, is that of collinearity, in particular, when do points lie on the same line. In particular, we shall

Thus, you should create a `Line` class, and associated to the line, there should be the following data

- `equation` this will be the equation of the line, when creating a point, we should check to see which lines it falls upon
- `points` this will be the set of points that lie on this line
- `name` this will be the name of this line

- `constructed` this is a boolean that will denote whether or not this line was constructed or given from the statement of the problem
- `index` this is a global index for this line
- `depth` this is a marker to denote in what order this line was created

Combinatorially, the worst case scenario will occur if no three points are collinear. Thus, every two points will form a line; thus, there will be $\binom{n}{2} = O(n^2)$ lines.

5.3. **Segments.** The idea behind segments is the same as lines; however, we use the segments to measure the distance between two points, so we will create a segment for every pair of points.

Thus, you should create a `Segment` class. These should have the following data associated to them

- `name` this is just the name of the segment
- `points` this will have the two points that determine the segment
- `length` this is the numeric quantity of the length of the segment (we shall use this as a hash to conjecture possible `cong` relations)
- `index` this is a global index for this segment (we shall use these segments and these indices in AR)
- `constructed` this is a boolean
- `depth` this a marker to denothe in what order this segment was created

Combinatorially, we have that every two distinct points determines a segment, so we will have $\binom{n}{2} = O(n^2)$ points.

5.4. **Pairs of Segments.** Once we have segments, there will be times where we wish to look at the ratio between two segments. Such computations are useful when determining the `eqratio` or `rcompute` predicate. Now instead of computing all the ratios at all times. We can precompute the ratios between two segments, and then if we hash these pairs by the numeric value of the ratio, then when creating new pairs of ratios, we can check to see if the hash values match

5.5. **Circles.** There are two ideas behind that of circles, namely `circle O A B C` which denotes circumcircles, and `cyclic A B C D` which denotes points lying on the same circle. The downside is that `cyclic` may doesn't capture the center of the circle, so there is also the variant of `cyclic_with_center` which captures the center.

5.6. **Angles.**

5.7. **Triangles.** FINISH WRITNG THE REMAINDER OF THE GEOMETRIC OBJECTS

6. Basic Predicates

In this section, we will discuss the basic predicates that we will use in our geometry system, and how we will choose to store this data. The general outline is as follows: our system will use predicates of the form `col A B C`, `eqangle A B C P Q R`, `cong A B C D`, etc.. In the above, each of the terms `col`, `eqangle`, and `cong` are **predicates** that correspond to some geometric property in this case collinear points, equality of angles, and congruence of segments respectively. While `A B C D P Q R` all represent **points** in the problem statement. There are a few predicates which also involve other variables, but we will discuss those as they come up. Thus, a predicate consists of some word that signifies the geometric relation followed by some points. The word will signify the relation that the points satisfy.

We remark that we build everything based off of the points of the problem, and the points will be the geometric objects that we consider when dealing with predicates. While it may be possible to have predicates that take in other geometric objects such as lines, triangles, circles, etc., including such information in the predicates might complicate things. Thus, we build everything off of points; however, our other geometric objects are used for generating conjectures.

Once we have points, we can start putting them together into predicates such as `col A B C`. We shall make a class for each of the different kinds of predicates. However, there are a lot of shared attributes of each kind of predicate. Thus, we shall create a class called `RelationNode` which is the abstraction of all the different kinds of predicates. Furthermore, each `RelationNode` can be thought of as the vertex in the proof graph.

The `RelationNode` class should have the following data associated to it:

- `name` this will be the name of the node

- `type` this tells us what predicate the node should be.
- `index` this should be
- `given` this is a Boolean which is true if the predicate was given in the problem statement, and false if it was deduced
- `parents` this will be a list of other predicates where if the given node was deduced from other information then the parents describe the geometric information used in deducing the given predicate
- `rule` this will be a string which says by which rule (or if AR was used) in the deduction
- `statement` this will be a string used when outputting the proof that will state how the relation is known. Whether it is given, deduced in DD (from which rule and what the parents were), or deduce from AR (explaining the linear combination used in the deduction).

The proof graph will be a directed graph whose vertices correspond to `RelationNodes`, and whose edges correspond to the `parents` of the given nodes. Thus, the `parents` data should be considered as directed edges for a given node pointing to where it came from. Using this structure, we have that FIGURE OUT WRITING THE REST OF THIS

6.1. **coll.**

6.2. **cong.**

6.3. **cyclic.**

6.4. **cyclic_with_center.**

6.5. **circle.**

6.6. **eqangle.**

6.7. **para.**

6.8. **perp.**

6.9. **midp.**

6.10. **eqratio.**

6.11. **simtri1(2).**

6.12. **contri1(2).**

6.13. **acompute.**

6.14. **rcompute.**

6.15. **distmeq.**

6.16. **distseq.**

6.17. **angeq.**

6.18. **sameclock.**

6.19. **sameside.**

6.20. **overlap.**

7. Problem Data Structure

Now we will discuss the Problem Data Structure. This will be the contents of the `Problem.py` file. In particular, this

8. Deductive Database

Now the **Deductive Database (DD)** is one of the more basic ideas of AlphaGeometry. The general idea of DD is that it is a list of rules where we.

9. Generating Conjectures

10. Algebraic Reasoning

10.1. **Angle Matrix.**

10.1.1. *Measure of a Line with Respect to the x-axis.* In this section, we discuss a different way to calculate $\angle_1$. This is not a different angle convention, but rather a way that w

10.2. **Ratio Matrix.**

11. The Basic Solver

At this point, we have now developed a basic geometry solver. The loop and algorithm to solve this

12. Implementing Area

We shall now discuss the issue of area. The classical versions of AlphaGeometry and AlphaGeometry2 do not consider the concept of area. In doing so, these systems can't deduce the Pythagorean theorem. In this section, we will discuss what additions and complications come up when trying to define area.

For the highly motivated students, you may try to implement the concept of area into your system (consider it an extra-credit project or a challenge for those who are interested).

12.1. **Area Matrix.**

13. Auxiliary Constructions

The goal of this section is to discuss the various kind of rabbits that we might consider. We shall create a file called `constructions.py` which will do these geometric constructions and .

14. Generating Synthetic Data

Now that we have a basic solver, and a way to do constructions. The goal now is to come up with data that will then be used to

15. Cleaning the Data

In this section, we will briefly discuss the process of cleaning the synthetic data. In particular, it might be the case that when generating the synthetic data, you may have two data points which appear to be different, but are actually the same. What we mean by this is that the two data points might have for example `col A B C` and `col X Y Z` in the assumptions, and after doing the transformation $A \mapsto X$, $B \mapsto Y$, and $C \mapsto Z$, then the two data points may actually be the same.

Another way in which two data points might look different, but are actually the same, is that the predicates might appear in a different order. That is `col A B C; col C D E...` should be the same as `col C D E; col A B C...`

Thus, for the synthetic data, even though the points that appear FINISH WRITING THIS

16. Fine Tuning the Language Model

In the original run of this course, we had an engineer from Google DeepMind give a guest lecture on fine-tuning the language model. In particular, we had used Gemma (documentation linked here) as the language model in which we had finetuned. For those who have more experience with language models and want to explore/experiment with others, feel free to use Hugging Face.

In particular, the hardest part of fine-tuning the language model was just coming up with the data to tune the model on. Once, you have the data, then fine tuning is as simple as following a tutorial such as this one for Gemma (tutorial linked).

For those at Brown, we recommend doing the finetuning on OSCAR (Brown's super computer).

17. n-Shot Prompting

An alternative to fine-tuning a language model which also can solve the problem, is the idea of prompt engineering, and n-shot prompting. The benefit to n-shot prompting is that it requires a smaller amount of synthetic data, and it is easier to change which language model you use (without having to fine-tune another model).

For n-shot prompting, you will write a preset prompt to give the language model (which will only change with the given problem). In addition, you will also give the language model n examples of input/output pairs from your synthetic data, then the language model will give its guess to the correct output.

One advantage that this has is that you may feed the language model more context (such as a natural language statement); however, we remark that this strategy has the downside is that you are feeding the language model a larger prompt, so it will require more tokens to run compared to a fine-tuned model where you just feed in the formalization.

Here is a rough outline of what a prompt may look like:

" You are a Euclidean geometry expert that solved IMO geometry problems. You must give a geometric construction that is useful in solving the following geometry problem `{natural language statement}`.

We use the following system to formalize the problem... (here you will give an explanation of the predicates and their meanings)

Furthermore, you may suggest the following constructions... (here you will give a description of the constructions in your system)

We have currently done the following constructions; thus far `{constructions perfomred}`.

A formalized version of the current state of the problem after finding all possible deductions is given by `{Formal State of the Problem}`.

Here is an example of n examples of input problem states, and output construction suggestions that have worked for previous problems `{List of n Examples}`.

Give the construction in the required output format. "

Once you have your prompt, you can just incorporate this LLM call into the main loop of your solver when you get stuck to suggest a construction.

18. Integrating the Language Model

Now that we have discussed the way in which we can use language models to help solve our math problems, we can now integrate them into the solving loop. In particular, you might consider having different modes for your system to run on.

(1) Basic Mode: This is the mode as described in Section 11 where you just run your deduction system to see if it solves the problem.
(2) Random Construction Mode: This mode will run the Basic solver until either it can't deduce anything more, or it has gone through enough loops of the deduction system. At that point, a random construction will be applied, and new conjectures will be formed, and then the basic solver will be run again (until a threshold of a given number of random constructions are done), or if the problem is solved.
(3) Expert Mode: This mode is like random; however, rather than applying a random construction. It will specifically apply constructions that are related to a given triangle (i.e. for a given triangle construct its circumcenter, orthocenter, excenters, etc.). The idea is that these points are useful for a triangle to argue around, so just do them all for a triangle, and hopefully this will do better than just a random construction.
(4) Fine-tuned Mode: This mode will prompt the fine-tuned language model if it is unable to deduce the goal.
(5) n-Shot Prompting Mode: This mode will prompt the language model using the idea of prompt engineering and n-shot prompting.

19. Automatic Diagram Generation and Autoformalization

This brief section is about incorporating an automatic diagram generation and/or autoformalization into your system. In the initial run of the course, we did not include this material. However, we shall include a

section for students who wish to incorporate it (or part of it) into their systems. In particular, rather than building a diagram generator from scratch, each team may search for some automatic diagram generators of that currently exist. We shall detail the pros and cons of incorporating this into your system, and a few automatic diagram generators that currently exist

Pros:

- Automatic diagram generation and autoformalization will allow for a single pipeline where you will input a natural language statement and then the system will produce a proof
- Simpler for the user of the system
- You can generate multiple diagrams, and if the system proves the result, that helps insure that the proof need not rely on the diagram

Cons:

- The autoformalized or generated diagram may be incorrect leading to the system having issues solving the problem
- We have built our system based off reading in a geogebra `.ggb` file and scrapping this data, to incorporate this there may be a different structure that you will need to learn and then write code to integrate it into your system
- If the autoformalizer hallucinates it may either lead out a predicate that is needed to solve the problem, or add an additional predicate which is not necessarily given, but makes the problem trivial.

For automatic diagram generation, you may consider [KHS21] which has a github (click here to access) that you may integrate into your system. However, if there are other automatic diagram generators that you wish to find and use, feel free to use those.

For autoformalization, you may try to use a language model in which you could try to do n-shot prompting with examples (since there isn't enough data to fine tune a model), you could also try doing a loop with agents in which you have several LLMs, one formalizes, and then one checks the formalization, and then you can have an editor. This loop will decrease the chance of hallucination.

20. Power of a Point

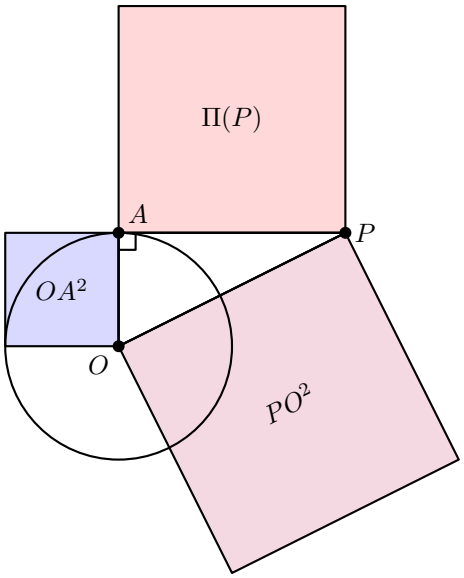
For the remainder of the notes (until we present the coding homeworks at the end of the document), we shall present some of the tools/theorems in Euclidean geometry. These concepts might not be used in your system, but you could try to either build your system in such a robust way that you can prove these theorems, or you could try to incorporate some of these theorems in your DDAR (although that might be a bit tricky).

We begin this section by discussing the **power of a point** which is a quantity that relates a point P and a given circle.

Definition 20.1. *The **power of a point** P with respect to a circle with center O and radius OA is the quantity:*

$$\Pi(P) = PO^2 - OA^2.$$

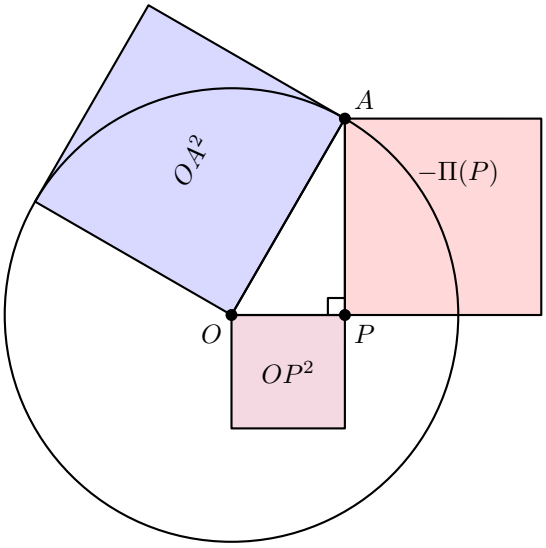
We remark that, even though our definition of the power of a point involves the point A. Since we just have the quantity OA^2 which is the radius of the circle squared. This quantity is independent of the point A chosen on the circle. In particular, it is most useful to pick a point A such that `perp P A A O`. Thus, we will have a right triangle, and can use the Pythagorean theorem. Thus, the power of a point P is the square of the distance of the tangent segement from P to the circle.



However, we now see an issue with this visualization of the power of a point. In particular, it depended upon the point P being located outside of the circle. Since a point inside the circle will not have a tangent to the circle. The definition allows for points inside of the circle. We see that

$$\Pi(P) : \begin{cases} > 0 & \text{P is outside the circle} \\ < 0 & \text{P is inside the circle} \\ = 0 & P \text{ is on the circle} \end{cases}$$

However, if P is inside the circle, then we can not pick a point A so that `perp P A A O`. To get around this, we can still make a right triangle, but picking A so that `perp O P P A`. Then we will have that $\Pi(P)$ is the negative area of the corresponding square. In particular, we can visualize this in the figure below.



We remark that due to its complex nature of involving a point, the center, and radius of a circle as well as its definition involving the non-linear operation of squaring. It isn't clear how to incorporate such a quantity in your AI. Furthermore, it is unclear what this quantity is actually measuring just from its definition. We now discuss a theorem that describes the power of point with respect to a circle to a ratio between either the secant line (in the case that the point is outside the circle), or the chord containing the point (in the case that the point lies inside the circle).

We remark that the statement of the theorem is different in the fact that the assumption lie in the angle part of AR while the conclusion lies in the ratio part of AR. Furthermore, as this is an implication rather than an if and only if statement, such a rule might be more appropriate for DD if you wish to implement it.

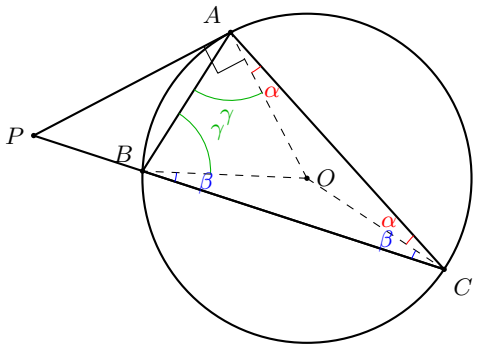
Proposition 20.2. `perp P A A O; cong O A O B; cong O A O C; col P B C⇒ PA^2 = P B · PC`.

In particular, we see that as $PA^2 = |\Pi(P)|^2$*, we can state the conclusion of the theorem as* $\Pi(P) = PB \cdot PC$*. In this case, we need not even have reference to the point* A *in the theorem statement.*

For P *outside the circle, this is the so called tangent-secant theorem.*

For P *inside the circle, this is the so called intersecting chords theorem.*

Proof. We consider the case where P is outside the circle. Draw the radii OA, OB, OC and the triangle ABC, as illustrated below.



Since $OA = OB = OC = r$ (all radii), the triangles OAC, OBC, and OAB are all isosceles. We label their base angles:

- $\triangle OAC$: $\angle OAC = \angle OCA = \alpha$,
- $\triangle OBC$: $\angle OBC = \angle OCB = \beta$,
- $\triangle OAB$: $\angle OAB = \angle OBA = \gamma$.

Since O lies inside $\triangle ABC$, the interior angles decompose as

$$\angle BAC = \alpha + \gamma, \qquad \angle ABC = \beta + \gamma, \qquad \angle BCA = \alpha + \beta.$$

Their sum equals π:

$$(\alpha + \gamma) + (\beta + \gamma) + (\alpha + \beta) = 2(\alpha + \beta + \gamma) = \pi,$$

so $\alpha + \beta + \gamma = \dfrac{\pi}{2}$.

Since PA is tangent to the circle, $PA \perp OA$, giving $\angle PAO = \dfrac{\pi}{2}$. Therefore,

$$\angle PAB = \angle PAO - \angle OAB = \frac{\pi}{2} - \gamma = (\alpha + \beta + \gamma) - \gamma = \alpha + \beta.$$

Since $\angle ACB = \alpha + \beta$ as well, we have $\angle PAB = \angle ACB = \angle ACP$.

Now consider triangles $\triangle APB$ and $\triangle CPA$. They share the angle at P, and $\angle PAB = \angle PCA = \alpha + \beta$. By AA similarity, $\triangle APB \sim \triangle CPA$. Reading off the resulting ratio from corresponding sides:

$$\frac{PA}{CP} = \frac{PB}{PA} \implies PA^2 = PB \cdot PC. \qquad \square$$

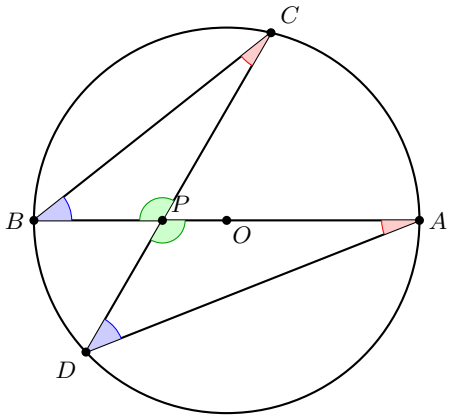
This concludes the proof when P lies outside of the circle.

We now turn to the case where P is inside the circle, giving rise to the **intersecting chords theorem**. In particular, we remark that the assumption of `perp P A A O` is superfluous, and it suffices to just prove the following.

Proposition 20.3 (Intersecting Chords)**.** *Let* P *be a point inside a circle with center* O*, and let two chords through* P *meet the circle at* A*,* B *and* C*,* D *respectively. Then*

$$PA \cdot PB = PC \cdot PD = |\Pi(P)|.$$

Proof. Choose AB to be the chord passing through both P and the center O, with A the farther endpoint from P, so the order along the line is B-P-O-A. Let CD be any other chord through P.



Since $OA = OB = r$, we have $PO + OA = PA$ and $PO - OA = PO - OB = -PB$. Therefore,

$$\Pi(P) = PO^2 - OA^2 = (PO - OA)(PO + OA) = (-PB)(PA) = -PA \cdot PB.$$

It suffices to show $PA \cdot PB = PC \cdot PD$, for then $\Pi(P) = -PC \cdot PD = -|\Pi(P)|$, giving $|\Pi(P)| = PC \cdot PD$.

Since A, B, C, D all lie on the circle, the inscribed angles $\angle PCB$ and $\angle PAD$ both subtend arc BD (red), so $\angle PCB = \angle PAD$; likewise $\angle PBC$ and $\angle PDA$ both subtend arc CA (blue). The angles $\angle CPB$ and $\angle APD$ are vertical angles at P (green). By AA similarity,

$$\triangle PCB \sim \triangle PAD.$$

Reading off corresponding sides:

$$\frac{PC}{PA} = \frac{PB}{PD} \implies PC \cdot PD = PA \cdot PB = |\Pi(P)|. \qquad \square$$

Remark 20.4. *We now remark on the notion of signed length. In particular, we see that in the case where P lies outside of the circle, we have that $\Pi(P)$ is positive, and $\Pi(P) = PB * PC$. However, in the case where P is inside the circle, we have that $\Pi(P)$ is negative, and so $\Pi(P) = PB * PC$ doesn't entirely make sense if we are measuring PB and PC to both be positive lengths. However, to make sense of this since P lies between the points B and C on the line, if we choose an orientation of the line, then one of these segments is positively orientated and the other is negatively orientated which explains why $\Pi(P)$ is negative for points inside the circle.*

21. Radical Axis

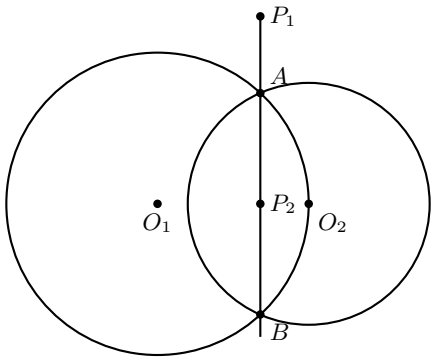
We shall now discuss the radical axis, this relates the power of points between two circles.

Definition 21.1. *The **radical axis** of two circles O_1 and O_2 (that may intersect) is the line l such that for any $P \in l$, we have that $\Pi_1(P) = \Pi_2(P)$ where $\Pi_i(P)$ is the power of the point P with respect to the circle O_i.*

We begin with some basic lemmas about the radical axis of two circles.

Proposition 21.2. *Assume that circles O_1 and O_2 intersect at two points A and B. Then the line AB is the radical axis.*

Proof. We show that every point P on line AB satisfies $\Pi_1(P) = \Pi_2(P)$, treating separately the cases where P lies outside versus inside the two circles.



Case 1: P_1 on line AB, outside both circles. The line through P_1, A, B is a secant to circle O_1 meeting it at A and B. By the power of a point (tangent-secant theorem applied to the secant P_1AB),

$$\Pi_1(P_1) = P_1A \cdot P_1B.$$

The same line is also a secant to circle O_2 meeting it at the same points A and B, so

$$\Pi_2(P_1) = P_1A \cdot P_1B.$$

Hence $\Pi_1(P_1) = P_1A \cdot P_1B = \Pi_2(P_1)$.

Case 2: P_2 on line AB, inside both circles. The segment AB is a chord of circle O_1 passing through P_2. By the intersecting chords theorem,

$$\Pi_1(P_2) = -P_2A \cdot P_2B.$$

The same chord lies inside circle O_2 as well, so

$$\Pi_2(P_2) = -P_2A \cdot P_2B.$$

Hence $\Pi_1(P_2) = -P_2A \cdot P_2B = \Pi_2(P_2)$.

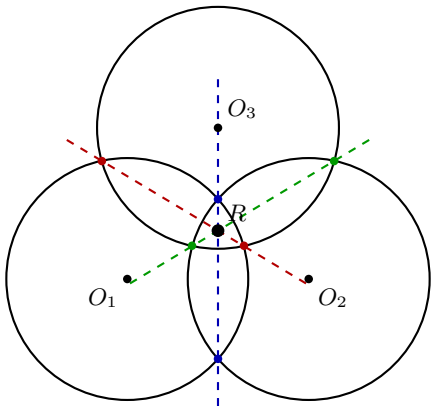
In both cases every point on line AB has equal power with respect to the two circles, so AB is the radical axis. □

We now state a theorem which describes the radical axes of three circles.

Theorem 21.3. *Let l_1, l_2, and l_3 be the radical axes of O_2 and O_3, O_1 and O_3, and O_1 and O_2 respectively. Then l_1, l_2, and l_3 are concurrent.*

Proof. Let P be on the intersection of l_1 and l_2. We claim that P is on l_3. Since $P \in l_1$, we have that $\Pi_2(P) = \Pi_3(P)$. Now since $P \in l_2$, we have that $\Pi_1(P) = \Pi_3(P)$. Thus, we will have that $\Pi_1(P) = \Pi_2(P) = \Pi_3(P)$. Thus, $P \in l_2$.

In the case where each pair of circles intersects at two points, the three radical axes are precisely the lines through each pair of intersection points, and their concurrency at the radical center R is illustrated below.



□

We remark that in the proof, we have assumed that the lines actually intersect. There is an edge case in which the radical axes are parallel. In this case, if we consider projective geometry, we can say that the lines are concurrent at infinity.

Furthermore, using this theorem one can construct the radical axis of two non-intersecting circles. We leave this as an exercise.

22. Menalaus's Theorem

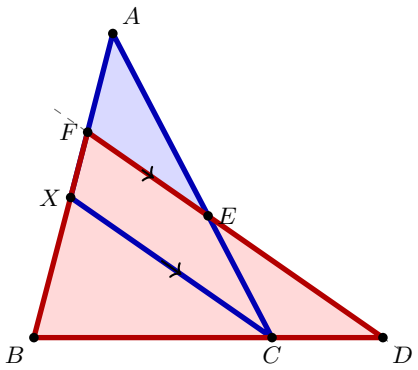
We now will give a proof of Menalaus' theorem.

Theorem 22.1 (Menalaus' Theorem)**.** `col A B F; col A C E; col B C D;`
Then `col D E F` $\Leftrightarrow$ $\frac{AF}{BF} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$
In particular, Menalus' theorem is saying that given a triangle ABC where we extend the lines defining the triangle. Then if we draw a line which intersects each of the three lines defininig the triangle at DEF, then we have a product of ratios is 1 *(and conversely given three points on these lines that satisfy this ratio, then they lie on the same line).*

Before proving Menalaus' Theorem, we remark that this theorem is another one in which it has a relationship between a collinear and a ratio. Thus, such a theorem combines the angle table in AR with the ratio table in AR. However, in the next section when we discuss Ceva's theorem, we will see a caveat as to why implementing this theorem is quiet messy.

Proof. We prove the forward direction: assuming D, E, F are collinear, we show $\dfrac{AF}{FB} \cdot \dfrac{BD}{CD} \cdot \dfrac{CE}{AE} = 1$.

Construction. Let X be the point on line AB such that $XC \parallel DEF$.



Step 1: $\triangle AXC \sim \triangle AFE$ **(blue triangle).** Since $XC \parallel FE$, we have $\angle AXC = \angle AFE$ (corresponding angles) and $\angle A$ is shared. By AA similarity, $\triangle AXC \sim \triangle AFE$, giving $AX/AF = AC/AE$. Since the order along line AB is A–F–X–B, we have $AX = AF + FX$ and $AC = AE + EC$, so

$$\frac{AF + FX}{AF} = \frac{AE + EC}{AE} \implies \frac{FX}{AF} = \frac{EC}{AE}.$$

Step 2: $\triangle BXC \sim \triangle BFD$ **(red triangle).** Since $XC \parallel FD$, we have $\angle BXC = \angle BFD$ (corresponding angles) and $\angle B$ is shared. By AA similarity, $\triangle BXC \sim \triangle BFD$, giving $BX/BF = BC/BD$. Since $BX = BF - FX$ and $BC = BD - CD$,

$$\frac{BF - FX}{BF} = \frac{BD - CD}{BD} \implies \frac{FX}{BF} = \frac{CD}{BD} \implies \frac{BD}{CD} = \frac{BF}{FX}.$$

Conclusion. Substituting into the product:

$$\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = \frac{AF}{FB} \cdot \frac{BF}{FX} \cdot \frac{FX}{AF} = 1.$$

Now for the reverse direction. We shall assume that $\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$. Now we shall take the points D and E, and extend this line to intersect AB at a point F'. Our goal is to prove that $F = F'$. In particular, since `col D E F'`, we have that

$$\frac{AF'}{F'B} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1.$$

Thus, we may conclude that

$$\frac{AF'}{F'B} = \frac{AF}{FB}.$$

Now we shall add $1 = \frac{F'B}{F'B} = \frac{FB}{FB}$ to both sides and use the fact that $AF' + F'B = AB$ and $AF + FB = AB$ to conclude that

$$\frac{AF' + F'B}{F'B} = \frac{AF + FB}{FB} \Rightarrow \frac{AB}{F'B} = \frac{AB}{FB} \Rightarrow F'B = FB.$$

Thus, as the distance from F to B and F' to B are the same and we have that both F and F' lie on the line AB, we may conclude that $F = F'$. □

We remark that there are really two choices of F' that are the same distance from B as F on the line AB; however, if one considers the signed distance this forces $F' = F$ in the above proof.

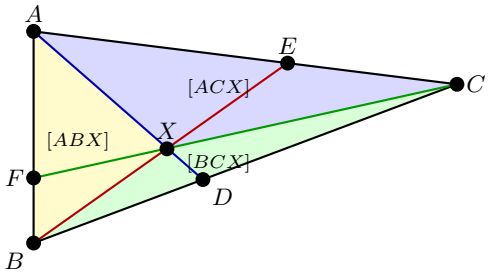
23. Ceva's Theorem

We now will give a proof of Ceva's theorem.

Theorem 23.1 (Ceva's Theorem)**.** `col A B F; col A C E; col B C D;`
Then `concurrent A D B E C F` $\Leftrightarrow$ $\frac{AF}{BF} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$

Remark 23.2. *We now discuss the reason Menalaus' and Ceva's theorems are difficult to implement. In particular, the statements of the two theorems are nearly identical. The only difference is that we have either:* `col D E F` *or* `concurrent A D B E C F`*. The difference between the two comes down to the notion of the signed distance. Since in the case of Menalaus' theorem, we will have that ratio of the signed distances will actually be* -1 *as two of the points lie inbetween the segments of the triangle.*

Proof. We begin with the forward direction. Suppose the lines AD, BE, and CF are concurrent at an interior point X. We will show $\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$. Denote the area of $\triangle XYZ$ by $[XYZ]$.



Claim 1: $\frac{BD}{CD} = \frac{[ABX]}{[ACX]}$.
Triangles $\triangle ABD$ and $\triangle ACD$ share the altitude from A to line BC, so

$$\frac{[ABD]}{[ACD]} = \frac{BD}{CD}.$$

Triangles $\triangle XBD$ and $\triangle XCD$ share the altitude from X to line BC, so

$$\frac{[XBD]}{[XCD]} = \frac{BD}{CD}.$$

Since X lies on segment $\overline{AD}$, we have $[ABD] = [ABX] + [XBD]$ and $[ACD] = [ACX] + [XCD]$. Subtracting:

$$\frac{BD}{CD} = \frac{[ABD] - [XBD]}{[ACD] - [XCD]} = \frac{[ABX]}{[ACX]}.$$

Claim 2: $\frac{CE}{AE} = \frac{[BCX]}{[ABX]}$.

Applying the same argument to the line $\overline{BE}$, with X on segment $\overline{BE}$ and E on $\overline{CA}$: triangles $\triangle BCE$ and $\triangle ABE$ share the altitude from B to CA, while $\triangle XCE$ and $\triangle XAE$ share the altitude from X to CA. Since $[BCE] = [BCX] + [XCE]$ and $[ABE] = [ABX] + [XAE]$:

$$\frac{CE}{AE} = \frac{[BCE] - [XCE]}{[ABE] - [XAE]} = \frac{[BCX]}{[ABX]}.$$

Claim 3: $\dfrac{AF}{FB} = \dfrac{[ACX]}{[BCX]}$.

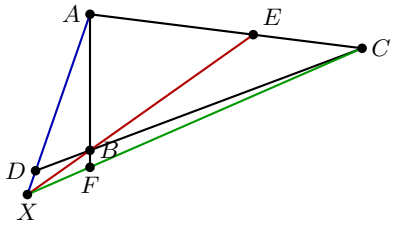
Applying the same argument to the line $\overline{CF}$, with X on segment $\overline{CF}$ and F on $\overline{AB}$: triangles $\triangle ACF$ and $\triangle BCF$ share the altitude from C to AB, while $\triangle XAF$ and $\triangle XBF$ share the altitude from X to AB. Since $[ACF] = [ACX] + [XAF]$ and $[BCF] = [BCX] + [XBF]$:

$$\frac{AF}{FB} = \frac{[ACF] - [XAF]}{[BCF] - [XBF]} = \frac{[ACX]}{[BCX]}.$$

Multiplying the three claims, the areas telescope:

$$\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = \frac{[ACX]}{[BCX]} \cdot \frac{[ABX]}{[ACX]} \cdot \frac{[BCX]}{[ABX]} = 1. \qquad \square$$

The proof above covers the case where D, E, F all lie on the sides of $\triangle ABC$, so the concurrency point X lies *inside* the triangle. The same argument applies when some of these points lie on extensions of the sides, in which case X lies *outside* the triangle.



Here D lies on the extension of $\overline{BC}$ beyond B, F lies on the extension of $\overline{AB}$ beyond B, and X lies outside $\triangle ABC$. In this configuration X lies beyond D on line AD (order: A–D–X), so the area decomposition in each claim *adds* instead of subtracts. For Claim 1, since $[ABX] = [ABD]+[XBD]$ and $[ACX] = [ACD]+[XCD]$:

$$\frac{BD}{CD} = r \implies \frac{[ABX]}{[ACX]} = \frac{[ABD] + [XBD]}{[ACD] + [XCD]} = \frac{r\,[ACD] + r\,[XCD]}{[ACD] + [XCD]} = r,$$

and analogous additions hold in Claims 2 and 3. The telescoping product is identical, so $\frac{AF}{FB} \cdot \frac{BD}{CD} \cdot \frac{CE}{AE} = 1$ in both cases.

Now for the converse. The proof is analogous to the converse of Menalaus' theorem. In particular, if we have that BE and CF intersect at the point X. Then if we draw the line from AX and say it intersects at the point D' on the line BC. The claim will be that $D = D'$. From the forward direction of Ceva's theorem since the lines are concurrent, we have that

$$\frac{AF}{FB} \cdot \frac{BD'}{CD'} \cdot \frac{CE}{AE} = 1.$$

Now in a similar way to the converse of Menalaus' theorem, we will havve that

$$\frac{BD}{DC} = \frac{BD'}{D'C}.$$

Thus, in a similar way to the proof of Menalaus' theorem, we will have that $D = D'$.

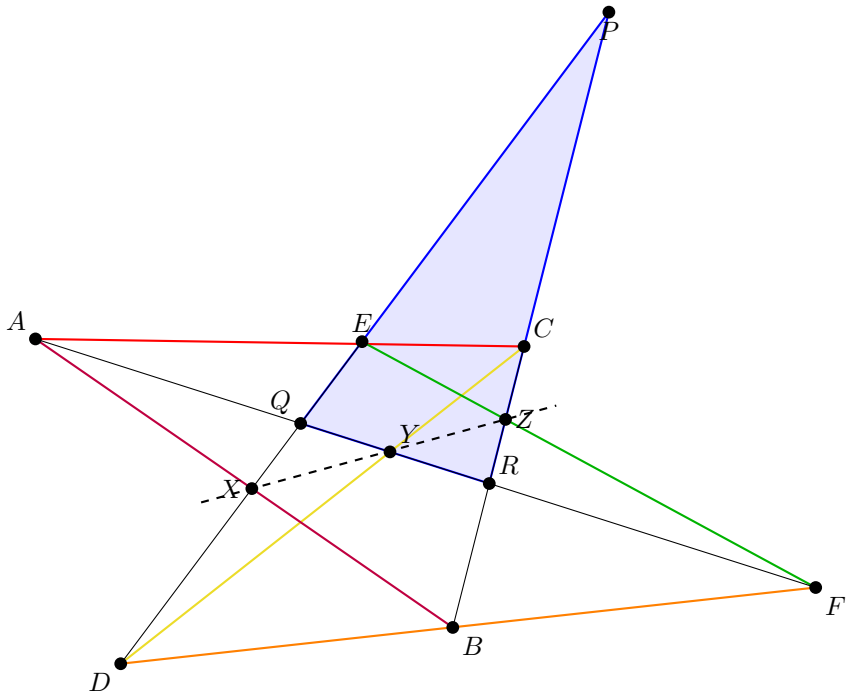
24. Pappus's Theorem

We now prove Pappus's Hexagon Theorem.

Theorem 24.1 (Pappus's Hexagon Theorem)**.** *Given* `col A E C; col D B F`*, then draw the "hexagon"* `ABCDEF`*, and let X, Y, and Z be the intersections of the opposite sides of the hexagon, then* `col X Y Z`*.*

Proof. We consider the configuration where the line DE is not parallel to the line BC. Let P be the intersection of DE with BC. Let Q be the intersection of DE with AF. Let R be the intersection of BC with AF.

We see this in the figure below:



We apply Menelaus's Theorem to the central triangle $\triangle PQR$ (highlighted blue) using the five specified transversal lines (AEC, DBF, DYC, EZF, and AXB). Because these lines intersect the extended sides of $\triangle PQR$, the product of the directed ratios of the segments equals -1.

(1) Line AEC (red line) intersecting $\triangle PQR$:

$$\frac{PE}{EQ} \cdot \frac{QA}{AR} \cdot \frac{RC}{CP} = -1$$

(2) Line DBF (orange line) intersecting $\triangle PQR$:

$$\frac{PD}{DQ} \cdot \frac{QF}{FR} \cdot \frac{RB}{BP} = -1$$

(3) Line DYC (yellow line) intersecting $\triangle PQR$:

$$\frac{PD}{DQ} \cdot \frac{QY}{YR} \cdot \frac{RC}{CP} = -1$$

(4) Line EZF (green line) intersecting $\triangle PQR$:

$$\frac{PE}{EQ} \cdot \frac{QF}{FR} \cdot \frac{RZ}{ZP} = -1$$

(5) Line AXB (purple line) intersecting $\triangle PQR$:

$$\frac{PX}{XQ} \cdot \frac{QA}{AR} \cdot \frac{RB}{BP} = -1$$

We observe that equations 3, 4, and 5 contain the specific segment ratios $\frac{QY}{YR}$, $\frac{RZ}{ZP}$, and $\frac{PX}{XQ}$ needed to establish the collinearity of X, Y, and Z (dotted line) with respect to $\triangle PQR$.

To isolate these terms, we multiply equations 3, 4, and 5 together, and then multiply by the inverses of equations 1 and 2:

$$\left(\frac{PD}{DQ}\cdot\frac{QY}{YR}\cdot\frac{RC}{CP}\right)\left(\frac{PE}{EQ}\cdot\frac{QF}{FR}\cdot\frac{RZ}{ZP}\right)\left(\frac{PX}{XQ}\cdot\frac{QA}{AR}\cdot\frac{RB}{BP}\right)\left(\frac{EQ}{PE}\cdot\frac{AR}{QA}\cdot\frac{CP}{RC}\right)\left(\frac{DQ}{PD}\cdot\frac{FR}{QF}\cdot\frac{BP}{RB}\right)=-1$$

Regrouping the fractions to cancel corresponding terms:

$$\left(\frac{PD}{DQ}\cdot\frac{DQ}{PD}\right)\left(\frac{RC}{CP}\cdot\frac{CP}{RC}\right)\left(\frac{PE}{EQ}\cdot\frac{EQ}{PE}\right)\left(\frac{QF}{FR}\cdot\frac{FR}{QF}\right)\left(\frac{QA}{AR}\cdot\frac{AR}{QA}\right)\left(\frac{RB}{BP}\cdot\frac{BP}{RB}\right)\left(\frac{PX}{XQ}\cdot\frac{QY}{YR}\cdot\frac{RZ}{ZP}\right)=-1$$

All of the intermediate ratios cancel perfectly to 1, leaving only:

$$\frac{PX}{XQ}\cdot\frac{QY}{YR}\cdot\frac{RZ}{ZP}=-1$$

By the converse of Menelaus's Theorem, because the product of the directed ratios along the sides of $\triangle PQR$ equals -1, the points X, Y, and Z must be collinear. □

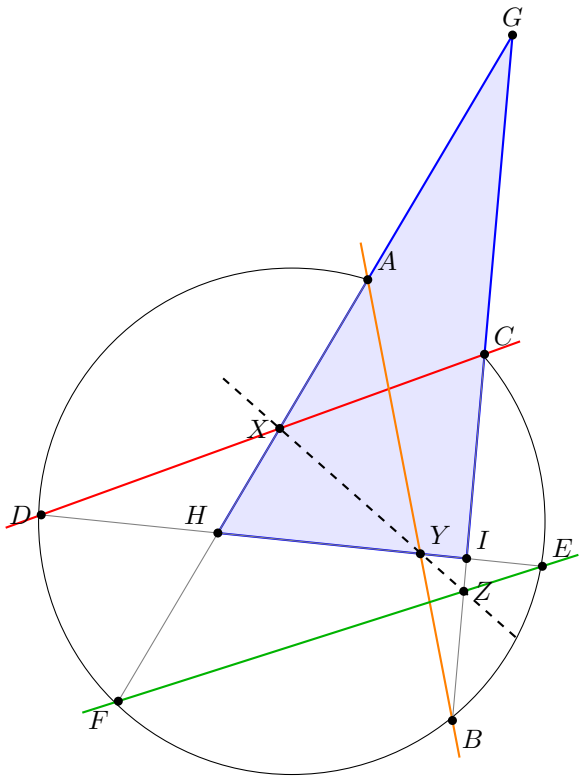
Remark 24.2. *We remark that the proof we have provided uses the point P which is the intersectoin of DE and BC, and when AB is parallel to EF; however, if these pairs of lines are parallel, then the proof which we have provided does not work. We leave this case as an exercise to the reader.*

25. Pascal's Theorem

We now prove Pascal's Hexagon Theorem.

Theorem 25.1 (Pascal's Hexagon Theorem)**.** *Assume that a hexagon $ABCDEF$ is inscribed in a circle and let X, Y, Z be the intersections of opposite sides, then* `col X Y Z`*.*

Proof. Given the hexagon $ABCDEF$ inscribed in a circle, we let G be the intersection of lines AF and BC, H be the intersection of lines AF and DE, and I be the intersection of lines BC and DE. This forms the (blue) triangle $\triangle GHI$. We observe this in the diagram below:



We now apply Menelaus's Theorem to the transversal lines DXC (red line), AYB (orange line), and EZF (green line) with respect to the (blue) triangle $\triangle GHI$. Because these transversals intersect the extended sides of the triangle, the product of their directed ratios is -1:

(1) (Red) Line DXC intersecting $\triangle GHI$:

$$\frac{GC}{CI} \cdot \frac{ID}{DH} \cdot \frac{HX}{XG} = -1$$

(2) (Orange) Line AYB intersecting $\triangle GHI$:

$$\frac{HA}{AG} \cdot \frac{GB}{BI} \cdot \frac{IY}{YH} = -1$$

(3) (Green) Line EZF intersecting $\triangle GHI$:

$$\frac{HF}{FG} \cdot \frac{GZ}{ZI} \cdot \frac{IE}{EH} = -1$$

We can see that the ratios $\frac{HX}{XG}$, $\frac{IY}{YH}$, and $\frac{GZ}{ZI}$ are the exact three components we need to apply the converse of Menelaus's theorem for the line XYZ with respect to $\triangle GHI$.

To isolate these terms, we multiply all three equations together. Grouping the intersections G, H, and I with their corresponding intersecting chords on the circle yields the following equations by the Power of a Point (Intersecting Chords) Theorem:

$$\frac{DI \cdot IE}{BI \cdot IC} = 1$$
$$\frac{BG \cdot GC}{AG \cdot GF} = 1$$
$$\frac{AH \cdot HF}{DH \cdot HE} = 1$$

Thus, when we multiply the three Menelaus equations together and apply these power of a point identities, the lengths of the circle's chords cancel out perfectly to 1:

$$\left(\frac{GC \cdot GB}{AG \cdot FG}\right)\left(\frac{HA \cdot HF}{DH \cdot EH}\right)\left(\frac{ID \cdot IE}{CI \cdot BI}\right)\left(\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI}\right) = -1$$

$$(1) \cdot (1) \cdot (1) \cdot \left(\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI}\right) = -1$$

This simplifies to:

$$\frac{HX}{XG} \cdot \frac{IY}{YH} \cdot \frac{GZ}{ZI} = -1$$

By the converse to Menelaus's theorem, because the product of the directed ratios along the sides of $\triangle GHI$ equals -1, we conclude that the points X, Y, and Z must be collinear. □

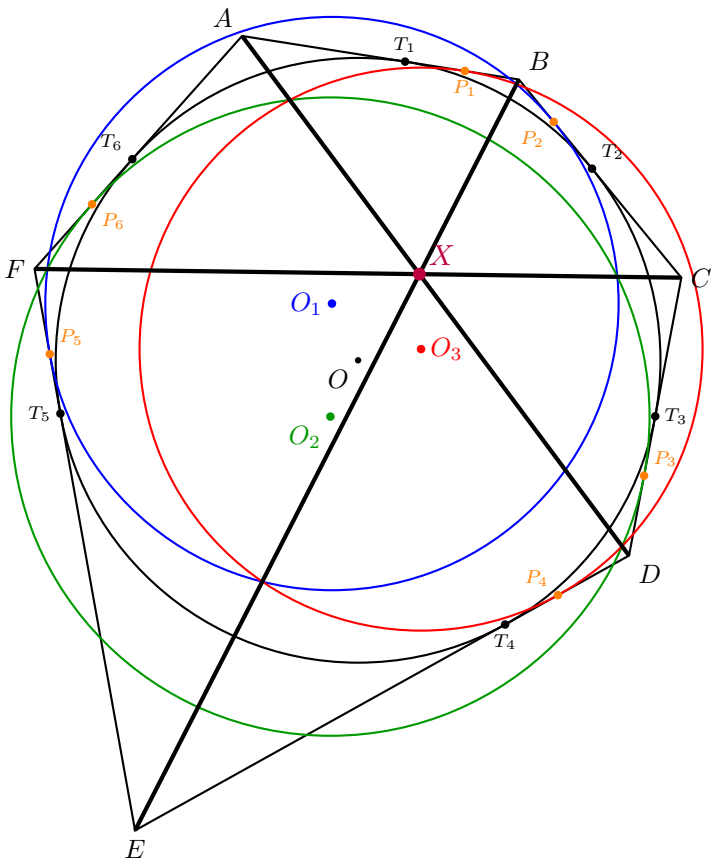
Remark 25.2. *Similar to Pappus's theorem, our proof relied on a simplification where we had the intersection points of G, H, and I. If the corresponding lines are parallel, then the intersections are points at inifinity, and the theorem is still true, but an additional proof is necessary.*

26. Brianchon's Theorem

We will now discuss Brianchon's theorem.

Theorem 26.1. *Assume that a circle is inscribed in a hexagon $ABCDEF$, then AD, BE, and CF are concurrent.*

Proof. Let the hexagon $ABCDEF$ be circumscribed around a circle ω with center O (black circle). Let the points of tangency of ω with the sides AB, BC, CD, DE, EF, and FA be T_1, T_2, T_3, T_4, T_5, and T_6 respectively.



Because the sides of the hexagon are tangent to the inscribed circle, the tangent segments drawn from each vertex to the incircle are equal in length. We denote these lengths as:

$$a = AT_1 = AT_6$$
$$b = BT_1 = BT_2$$
$$c = CT_2 = CT_3$$
$$d = DT_3 = DT_4$$
$$e = ET_4 = ET_5$$
$$f = FT_5 = FT_6$$

To construct our three specific circles, we shift the points of tangency by a small, arbitrary distance $k > 0$ along the perimeter of the hexagon. We plot six new points, P_1 through P_6 (labeled in orange), on the sides by alternately adding and subtracting k:

- On side AB, plot P_1 such that $AP_1 = a + k$ and $BP_1 = b - k$.
- On side BC, plot P_2 such that $BP_2 = b - k$ and $CP_2 = c + k$.
- On side CD, plot P_3 such that $CP_3 = c + k$ and $DP_3 = d - k$.
- On side DE, plot P_4 such that $DP_4 = d - k$ and $EP_4 = e + k$.
- On side EF, plot P_5 such that $EP_5 = e + k$ and $FP_5 = f - k$.
- On side FA, plot P_6 such that $FP_6 = f - k$ and $AP_6 = a + k$.

By alternating the arithmetic sign of the shift k, the distances from each vertex to its two adjacent offset points remain perfectly equal:

$$AP_1 = AP_6, \quad BP_1 = BP_2, \quad CP_2 = CP_3, \quad DP_3 = DP_4, \quad EP_4 = EP_5, \quad FP_5 = FP_6$$

We now define three auxiliary circles C_1 (blue), C_2 (green), and C_3 (red) that are tangent to opposite sides of the hexagon at these new shifted points:

- Let C_1 be the (blue) circle tangent to BC at P_2 and tangent to EF at P_5.
- Let C_2 be the (green) circle tangent to CD at P_3 and tangent to FA at P_6.
- Let C_3 be the (red) circle tangent to AB at P_1 and tangent to DE at P_4.

We remark that the reason that these circles exists is due to the uniform shift of the T_i. Since we have that given two lines and two points on those lines, then a circle exists which is tangent to both of these lines at the two specified points exists if and only if the distance from the point of intersection to the two specified points is the same. For example, we see that the circle C_1 exists as follows. We want C_1 to be tangent to BC and EF. Then if we let R be the point of intersection of BC and EF. Then since the circle ω is tangent to BC and EF at T_2 and T_5. We know that the $\overline{RT_2} = \overline{RT_5}$. Then we will have that $\overline{RP_2} = \overline{RP_5}$ since we will just be adjusting the distance by a factor of k from the intersection point.

We now find the radical axes for these three pairs of circles. The power of a point outside a circle is the square of the distance from the point to the circle's point of tangency.

(1) **Circles C_2 and C_3:** The power of vertex A to C_3 (red) is AP_1^2, and its power to C_2 (green) is AP_6^2. Since $AP_1 = AP_6$, vertex A has equal power to both circles. Similarly, the power of vertex D to C_3 (red) is DP_4^2, and its power to C_2 (green) is DP_3^2. Since $DP_3 = DP_4$, vertex D also has equal power to both circles. Therefore, the line connecting them, the diagonal AD, is the radical axis of C_2 and C_3.

(2) **Circles C_1 and C_3:** By the exact same logic, $BP_1 = BP_2$ and $EP_4 = EP_5$, meaning vertices B and E have equal power to C_1 (blue) and C_3 (red). Therefore, the diagonal BE is the radical axis of C_1 and C_3.

(3) **Circles C_1 and C_2:** Vertices C and F have equal power to C_1 (blue) and C_2 (green) because $CP_2 = CP_3$ and $FP_5 = FP_6$. Therefore, the diagonal CF is the radical axis of C_1 and C_2.

By the Radical Center Theorem, if three circles have pairwise radical axes, those three lines must either be parallel or intersect at a single concurrent point (the radical center). Since the three radical axes of C_1, C_2, and C_3 are exactly the principal diagonals AD, BE, and CF, we conclude that these diagonals are concurrent at the point X (purple), proving Brianchon's Theorem. □

27. Hilbert's Axioms

If there exists time at the end of the semester for additional lectures about mathematics beyond Euclidean geometry. We had discussed Hilbert's Axioms using Hartshorne's book [Har13]. Additional topics that could be discussed could also be hyperbolic/spherical geometry. We just refer to Hartshorne's book rather than recreating notes from scratch.

28. Homework

This section is meant to be the homeworks for the course. These sections describe what is to be done for each homework; however, on the GITHUB, there is a folder with the homework

28.1. **Homework 1 (Geometric Objects).** The first homework for the students

28.2. **Homework 2 (Predicates).**

28.3. **Homework 3 (AR).**

28.4. **Homework 4 (DD).**

28.5. **Homework 5 (Constructions).**

28.6. **Homework 6 (Conjectures).**

28.7. **Homework 7 (Data Generation).**

28.8. **Homework 8 (Implementation).**

28.9. **Homework 9 ().**

28.10. **Homework 10 ().**

29. Running AlphaGeometry on Windows

Before building our own system, it is useful to run a working reference implementation. The official AlphaGeometry release is difficult to install on Windows because it depends on the JAX/Flax/TensorFlow stack. We will instead use AlphaGeometryRE, a re-engineered fork that replaces the JAX-based language-model inference code with ChatLLM.cpp, a small native C++ backend with prebuilt Windows DLLs.

This section is deliberately practical. The source-code explanation begins in the next section.

Goals for this lab section

- Install AlphaGeometryRE on a Windows machine.
- Run one DD+AR-only proof.
- Run one LM-assisted proof.
- Recognize the moment when the language model contributes an auxiliary point.

29.1. Prerequisites.

- Windows 10 or 11.
- Python 3.10 or later, available on your `PATH` as `python`.
- `git` (optional; you may instead download the repository as a ZIP file from GitHub).

Unlike the original release, you do **not** need CUDA, JAX, or TensorFlow. The language model runs on the CPU through a small native library.

29.2. Step-by-Step Installation.

(1) **Get the source code.** Either clone the repository or download it as a ZIP from GitHub and extract it.

```
git clone https://github.com/foldl/AlphaGeometryRE.git
cd AlphaGeometryRE
```

(2) **Install the ChatLLM.cpp DLLs.** Download the prebuilt DLLs for your platform from the chatllm.cpp releases page (or build `libchatllm` from source following the binding instructions). Copy *all* of the downloaded `.dll` files into `src\chatllm\bindings`. There are several `ggml-cpu-*.dll` variants bundled together, one per CPU microarchitecture; the library picks the right one automatically at runtime, so do not skip any of them.

(3) **Install the Python dependencies.**

```
pip install -r requirements.txt
```

As mentioned in the introduction, you may optionally create a virtual environment first.

(4) **Run the test suite.** This both checks that your installation works and automatically downloads the language model the first time it is needed (it is fetched into `src\chatllm\quantized`).

```
run_tests.bat
```

29.3. Running a Problem.

The main entry point is `src\alphageometry.py`. It supports two solver modes, set with the `--mode` flag: `ddar` (the purely symbolic DD+AR solver) and `alphageometry` (DD+AR together with the language model, as discussed throughout these notes).

DD+AR only. The following solves IMO 2000 Problem 1 using only the deductive database and algebraic reasoning, with no language model involved:

```
python src\alphageometry.py ^
  --problems_file=examples\imo_ag_30.txt ^
  --problem_name=translated_imo_2000_p1
```

DD+AR with the language model. The `orthocenter` problem from `examples\examples.txt` (the same example we have used throughout these notes, see Section 1) is a case where DD+AR alone gets stuck and needs a rabbit from the language model:

```
python src\alphageometry.py ^
  --problems_file=examples\examples.txt ^
  --problem_name=orthocenter ^
  --mode=alphageometry ^
  --beam_size=2 ^
  --search_depth=2
```

What to watch for

The important story is not the timing information or the full proof log. Watch for three moments: DD+AR gets stuck, the LM proposes a new point, and DD+AR then proves the theorem after that point is added.

The relevant part of the output is short:

```
INFO - orthocenter
INFO - DD+AR failed to solve the problem.
...
INFO - LM output (score=-0.009415): "e : C a c e 02 C b d e 03 ;"
INFO - Translation: "e = on_line e a c, on_line e b d"

INFO - Solving: "a b c = triangle a b c; d = on_tline d b a c, on_tline d c a b; e =
    on_line e a c, on_line e b d ? perp a d b c"
...
INFO - Solved.
```

This is the neuro-symbolic loop from Section 1 in miniature. DD+AR gets stuck. The language model suggests the auxiliary construction `e = on_line e a c, on_line e b d`. After that point is added, DD+AR proves the theorem. The LM proposes a point; the symbolic system still proves the result.

29.4. **Command-Line Flags.** The most commonly used flags to `alphageometry.py` are:

- `--mode`: `ddar` or `alphageometry` (default `ddar`).
- `--problems_file`: path to a file of formalized problems. The two most useful examples are the short examples file and the IMO benchmark file in `examples`.
- `--problem_name`: name of the problem to solve within that file.
- `--defs_file` and `--rules_file`: paths to the construction definitions and DD deduction rules, defaulting to `data\defs.txt` and `data\rules.txt`.
- `--batch_size`, `--beam_size`, `--search_depth`: control the width and depth of the language model's proof search, analogous to the modes discussed in Section 18.
- `--out_file`: if given, writes the solution to this path in addition to printing it.

30. Reading AlphaGeometryRE

We now switch roles. The previous section was a lab guide: install the code and run a proof. This section is a guided reading of the same implementation. It remains at the end of the notes for now, but conceptually it belongs with the earlier sections on the proof graph, DD, AR, and the language model.

Guiding principle

The LM suggests auxiliary points. DD+AR proves or rejects them.

The rest of this section explains where that sentence appears in the code.

30.1. **Source Code Overview.** So far we have only run `src\alphageometry.py` as a black box. Before building your own version, it is worth a quick tour of `src`. This is a high-level orientation only; we return to several files in more detail below. (`extras` holds peripheral demo/visualization tools, not part of the core solver, and is not discussed here.)

File	Role in the system
`problem.py`	Parses the formal problem language into constructions, clauses, definitions, theorems, and dependencies.
`geometry.py`	Defines the node types used in the proof state: points, lines, circles, segments, directions, lengths, angles, and ratios.
`numericals.py`	Builds and checks a coordinate diagram. This helps avoid false or degenerate conjectures, but it is not the proof.
`graph.py` and `graph_utils.py`	Store the growing proof state: points, objects, known facts, algebra tables, and dependency records.
`dd.py`	DD, the deductive database. It applies rules from `data/rules.txt`.
`ar.py`	AR, algebraic reasoning. It turns angle, ratio, and distance facts into linear algebra.
`ddar.py`	The DD+AR loop: run DD, use AR consequences, repeat until the goal is proved or nothing new appears.
`trace_back.py`	Extracts a minimal proof from the dependency graph after the goal has been proved.
`pretty.py`	Converts formal dependencies into readable proof text.
`lm_inference.py`	Talks to the original purpose-trained AlphaGeometry-LM model through chatllm.cpp.
`lm_inference_openrouter.py`	Optional prompt-based backend for a hosted general-purpose model.
`alphageometry.py`	The command-line entry point and the full DD+AR+LM proof-search loop.

30.2. **The `data` Directory.** Two files in `data` are the system's entire symbolic knowledge of Euclidean geometry - `defs.txt` and `rules.txt`, both loaded once by `alphageometry.py` at startup. Neither file is Python code. Each is a small mathematical language: `defs.txt` says what constructions mean, and `rules.txt` says which theorems DD may use.

File	Question it answers
`defs.txt`	What does it mean to construct a point in this way?
`rules.txt`	Once some facts are known, what new facts may DD infer?

30.2.1. `rules.txt`*: the deduction rules used by DD.* Each line of `rules.txt` is one DD rule:

`premises => conclusion.`

The following #-comment gives the rule a human name. DD repeatedly looks for rules whose premises already hold and then adds the conclusion as a new fact.

- *equal-inscribed-angles-same-chord*

  `cyclic A B P Q` $\implies$ `eqangle P A P B Q A Q B`

  In words: if A, B, P, Q lie on a common circle, then the angle that chord AB subtends at P equals the angle it subtends at Q. This is exactly the Inscribed Angle Theorem.
- *SSS-congruence*

  `cong A B P Q`, `cong B C Q R`
  `cong C A R P`, `ncoll A B C` $\implies$ `contri* A B C P Q R`

  In words: if the three sides of triangle ABC are respectively congruent to the three sides of triangle PQR (and A, B, C are not collinear, i.e. actually form a triangle), then the two triangles are congruent. This is the SSS congruence criterion from classical geometry, written here as a single mechanical rule DD can apply.
- *parallel-from-double-perpendiculars*

  `perp A B C D`, `perp C D E F`
  `ncoll A B E` $\implies$ `para A B E F`

  In words: if line AB is perpendicular to CD, and CD is also perpendicular to EF, then AB is parallel to EF - two lines perpendicular to the same line are parallel to each other.

30.2.2. `defs.txt`*: the construction actions used to state problems.* Every construction you can write when formalizing a problem (recall `on_tline`, `midp`, `on_circle`, etc. from Section 1) is defined by a six-line block in `defs.txt`. For example:

```
midpoint x a b
x : a b
a b = diff a b
x : coll x a b, cong x a x b
midp a b
```

The six lines have fixed roles:

(1) `midpoint x a b` is the signature: construct a new point `x` from existing points `a` and `b`.
(2) `x : a b` records that `x` depends on `a` and `b`.
(3) `a b = diff a b` is a non-degeneracy condition: `a` and `b` must be distinct.
(4) `x : coll x a b, cong x a x b` gives the symbolic facts added to the proof state. A midpoint lies on line AB and satisfies $XA = XB$.
(5) `midp a b` names the numeric drawing routine in `numericals.py`.
(6) The blank line separates this definition from the next one.

The key distinction is between line 4 and line 5. Line 4 is what the prover is allowed to use in a proof. Line 5 is only how the program draws one generic diagram.

A second, shorter example, `on_circle`:

```
on_circle x o a
x : x o a
o a = diff o a
x : cong o x o a
circle o o a
```

This constructs a new point `x` lying on the circle centered at `o` through `a`; the only symbolic fact recorded is `cong o x o a` (`x` is the same distance from `o` as `a` is), which is exactly what it means for `x` to lie on that circle.

30.3. **A Closer Look at `dd.py`.**

Big idea

DD is theorem pattern matching.

A line in `rules.txt` is not just one theorem about one particular diagram. It is a reusable pattern. For example, the rule

$$\texttt{cyclic A B P Q} \implies \texttt{eqangle P A P B Q A Q B}$$

says: whenever the current problem contains four actual points playing the roles of A, B, P, Q, and those four points are known to be cyclic, DD may add the corresponding equal-angle statement. Thus `dd.py` is mainly a pattern matcher. It searches the current proof graph for all ways to replace the letters $A, B, P, Q, \ldots$ in a rule by the actual points in the problem.

There are two ways this matching is done:

- For about half of the rules, there is a hand-written matching function (collected in the dictionary `BUILT_IN_FNS`) that exploits the proof graph's internal indices - for example, points already grouped by which *direction* or *circle* they lie on - to jump straight to plausible matches instead of searching blindly. `match_cyclic_eqangle`, implementing the Inscribed Angle Theorem rule from Section 30.2, is one such function.
- Every other rule falls back to `match_generic`, a small general-purpose constraint solver: it looks up all currently known instances of each premise predicate (e.g. every known `coll` triple), then recursively tries to combine them into one consistent assignment of points to variables, backtracking whenever two premises disagree about what a shared variable should be.

Either way, the output is the same: a dictionary assigning the rule's variables to actual points. Once a rule has been matched, `bfs_one_level` does three things:

(1) **Match.** Find every match of every rule against the current proof state, as just described.

(2) **Justify.** For each match, double check that each premise is actually already known (not just numerically true), and record *why* - which earlier facts justify it - in a `Dependency` object. This bookkeeping is exactly what allows `trace_back.py` to later reconstruct a minimal proof.
(3) **Add.** Add the rule's conclusion to the proof graph as a new fact (skipping it if already known), and hand all newly added facts to AR, which may itself derive further facts algebraically before the next level begins.

This is why we write DD+AR, not DD then AR. A newly added DD fact is immediately sent to AR, and AR may produce more facts before the next DD level begins. The search is breadth-first: one full layer of consequences is processed before the next. Thus the depth counter really is a proof-depth counter, which is why `--search_depth` matters.

30.4. **A Closer Look at `ar.py`.**

Big idea

AR turns geometry into linear equations.

DD is good at applying named theorems. AR is good at chasing many equalities at once. Whenever the graph learns a new angle, ratio, or distance fact, AR translates it into a linear equation. Two quantities are then equal exactly when their algebraic expressions become identical. This is also a major speed gain: instead of rediscovering equality consequences by many small theorem applications, AR lets Gaussian elimination propagate whole families of equalities in one table.

There are three such tables, matching the three private attributes `atable`, `rtable`, `dtable` on the proof graph in `graph.py`:

- `AngleTable`: one variable per line *direction*, namely the value $d(\cdot)$ from Section 4.3, taken modulo π (`para`, `perp`, and `eqangle` facts all become linear equations among these directions). For instance, `eqangle P A P B Q A Q B` becomes, up to the modulo-π convention,

$$d(PA) - d(PB) = d(QA) - d(QB).$$

  Similarly, `para A B C D` says $d(AB) - d(CD) = 0$, while `perp A B C D` says $d(AB) - d(CD) = \pi/2$.

Remark: why modulo π?

A line, unlike a ray, has no preferred direction. Walking along AB toward A or toward B gives the same line. Therefore $d(AB)$ is only defined up to an added multiple of π. The `AngleTable` handles this by reducing the coefficient of `pi` modulo 1 whenever it checks whether an expression is zero:

```
>>> tb.modulo({'pi': -1})
{}
>>> tb.modulo({'pi': Fraction(1, 2)})
{'pi': Fraction(1, 2)}
```

A whole multiple of π disappears, while a genuine right angle $\pi/2$ remains. This does discard orientation: $d(AB)$ does not know which side of a line a point lies on, which is why orientation-sensitive statements need separate predicates such as `sameclock`. The diagram itself is not discarded; the code uses it once to choose the correct representative of an angle modulo π.

- `RatioTable`: one variable per segment *length*, but stored as its *logarithm*. Ratio facts such as $a/b = c/d$ are multiplicative, not linear - but $\log a - \log b = \log c - \log d$ is linear, so taking logarithms turns `cong` and `eqratio` facts into ordinary linear equations too.
- `DistanceTable`: one variable per (point, line) pair, namely that point's 1-dimensional coordinate along that specific line. This is what lets AR do intercept-theorem-style reasoning about several congruent or proportional segments lying on a common line. If $x_\ell(P)$ denotes the coordinate of point P along line ℓ, then a congruence such as $AB = CD$, with A, B on line ℓ and C, D on line m, becomes

$$x_\ell(A) - x_\ell(B) = x_m(C) - x_m(D),$$

  after choosing consistent orientations from the numeric diagram.

All three are built on the same generic `Table` class, which keeps a dictionary `v2e` ("variable to expression") expressing every known quantity as an affine combination of a *shrinking* set of free variables, plus a special constant (`pi` for `AngleTable`, `1` for the other two). Adding a new equality is one step of Gaussian elimination: if it only involves quantities already expressed in terms of the current free variables, one of those gets eliminated and re-expressed in terms of the rest (a pivot); if it mentions something new, that quantity simply gets expressed in terms of what is already known. Finding *every* new fact is then just comparing every pair of expressions and checking whether their difference collapses to a constant.

Remark: where is the Gaussian elimination?

AR does not first build one large matrix and then run Gaussian elimination from scratch. Instead, the table is kept in a reduced form as the proof develops. Each new geometric fact contributes one linear equation, and `add_expr` immediately performs the corresponding pivot: it solves for one variable in terms of the remaining free variables and substitutes that expression throughout `v2e`. This is why AR is fast. Equality consequences are propagated by algebraic normalization, rather than by many separate theorem applications. The separate matrix stored by `register` is mainly for proof traceback: later, a small linear program asks which input facts were responsible for a derived equality.

30.4.1. *Worked example: bisector and ex-bisector are perpendicular.*

The problem, in words

Let ABC be a triangle. Let D be its incenter and E the excenter opposite A. The line CD is then the *internal* bisector of $\angle ACB$, and the line CE is the *external* bisector of the same angle. Prove that $CD \perp CE$ - the internal and external bisectors of an angle are always perpendicular to each other.

The problem, formalized

This is exactly `test_ar.py`'s `test_angle_table_inbisector_exbisector` test, which uses the same construction vocabulary from Section 30.2:

```
a b c = triangle a b c; d = incenter d a b c; e = excenter e a b c
? perp d c c e
```

The diagram

Building the graph for this problem and calling the same drawing routine `alphageometry.py` uses internally produces Figure 1. The picture shows the incircle around D, the excircle around E, and the two segments CD, CE whose perpendicularity is exactly our goal.

Setting up the table

Rather than running full DD+AR, the test builds a bare `AngleTable` directly and feeds it three facts by hand, written using the directions $d(AC)$, $d(CD)$, $d(BC)$, $d(CE)$ from Section 4.3:

- *fact1*: CD bisects $\angle ACB$, i.e. the angle from AC to CD equals the angle from CD to BC.
- *fact2*: CE is the external bisector of $\angle ACB$.
- *fact3* (a distractor, deliberately unrelated to the perpendicularity claim): some other equal-angle fact about line AB, added only to confirm that AR's traceback correctly ignores it.

Watching the elimination happen, one fact at a time

This is the same incremental Gaussian elimination described above, but now traced through concretely. Before any facts are added, the table only knows about the constant:

```
v2e:  pi = pi
```

After *fact1* alone, $d(AC)$ is no longer free: the table has pivoted and re-expressed it in terms of $d(BC)$ and $d(CD)$, which remain free because nothing yet constrains them relative to each other:

```
v2e:  d(ac) = -d(bc) + 2*d(cd)
      d(bc) = d(bc)
      d(cd) = d(cd)
      pi    = pi
```

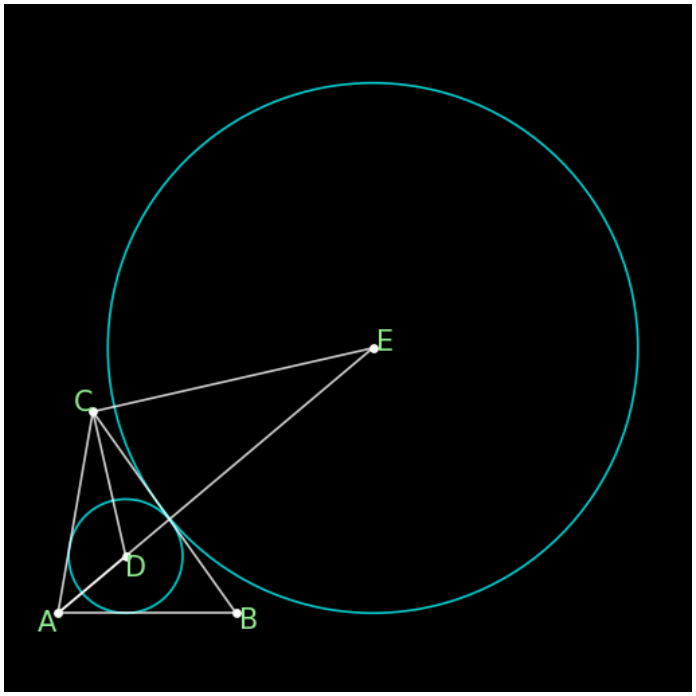


FIGURE 1. Triangle ABC with incenter D and excenter E (opposite A), drawn by `numericals.py`. CD is the internal bisector of $\angle ACB$; CE is the external bisector. The claim is $CD \perp CE$.

This already says something: $d(AC) = 2d(CD) - d(BC)$, i.e. $d(CD) = \frac{1}{2}(d(AC) + d(BC))$ - a bisector's direction really is the average of the two directions it bisects, and the table now carries that fact for every later computation involving $d(AC)$. After *fact2* is also added, $d(CE)$ becomes expressible too, this time in terms of $d(CD)$ alone plus a multiple of π:

```
v2e:  d(ac) = -d(bc) + 2*d(cd)
      d(bc) = d(bc)
      d(cd) = d(cd)
      d(ce) = d(cd) + -1/2*pi
      pi    = pi
```

This is already the answer, sitting in the table after only two facts: $d(CE)$ and $d(CD)$ differ by the constant $-\frac{1}{2}\pi$. Adding the distractor *fact3* pivots $d(AB)$ into the table as well, but - this is the point of including it - it does not touch the already-settled rows for $d(CD)$ or $d(CE)$ at all:

```
AR Table AngleTable (const=pi)
variables=6 eqs=12 groups=0
v2e (variable expansions):
  d(ab) = 3*d(bc) + -2*d(cd) + -pi
  d(ac) = -d(bc) + 2*d(cd)
  d(bc) = d(bc)
  d(cd) = d(cd)
  d(ce) = d(cd) + -1/2*pi
  pi = pi
```

The same final state, displayed as an explicit matrix (rows = every direction seen so far, columns = the three basis quantities the system has settled on - the constant `pi`, and the two directions `d(bc)` and `d(cd)` that turned out to be independent of everything else):

```
var        pi         d(bc)     d(cd)
----------------------------------------
```

d(ab)	-1/1	3/1	-2/1
d(ac)	.	-1/1	2/1
d(bc)	.	1/1	.
d(cd)	.	.	1/1
d(ce)	-1/2	.	1/1
pi	1/1	.	.

Every row is to be read as "(row variable) = (entry under pi)·π + (entry under d(bc))·$d(bc)$ + (entry under d(cd))·$d(cd)$", a dot meaning a zero coefficient; e.g. the d(ce) row reads $d(CE) = -\frac{1}{2}\pi + d(CD)$, matching the v2e line above it. Comparing the rows for d(cd) and d(ce): their difference is $d(CD) - d(CE) = \frac{1}{2}\pi$, a *constant*, independent of the two remaining free variables d(bc), d(cd) - whatever the actual triangle looks like, this difference never changes. A constant difference of exactly $\pi/2$ between two directions is precisely what it means for two lines to be perpendicular, so AR has discovered $CD \perp CE$ purely by linear algebra, without ever consulting a rule about bisectors.

Finding out *why*

The table above answers *whether* a fact is true, but proof output also needs to know *which* of the original input facts were actually responsible - exactly the role Dependency objects play for DD in Section 30.3. AR answers this with a second matrix, completely separate from v2e, by solving a small linear program over it. Section 30.5 works through exactly how this matrix is built and solved, first on a simple non-geometric example and then on this very problem.

30.5. Optional: Linear Programming and the *Why* Matrix.

Optional deep dive

This subsection first introduces linear programming as a mathematical topic in its own right. Only after that do we return to AR and explain why a linear program appears in proof traceback.

What is a linear program?

A *linear program* is an optimization problem in which both the quantity being optimized and the constraints are linear. In one common form, we choose a vector $x = (x_1, \ldots, x_n) \in \mathbb{R}^n$ and solve

$$\begin{aligned} \text{minimize} \quad & c^\top x \\ \text{subject to} \quad & Ax \le b, \\ & x \ge 0. \end{aligned}$$

The vector c gives the linear objective function, the inequalities $Ax \le b$ describe the feasible region, and the condition $x \ge 0$ usually means that the variables represent amounts of something: bags, hours, weights, probabilities, or, later, weights assigned to proof facts.

Geometrically, the feasible region is an intersection of half-spaces, hence a convex polygon in two variables, a convex polyhedron in three variables, and a convex polytope in higher dimensions (possibly unbounded). The key feature is that a linear objective has no hidden curvature. If an optimum exists, one can find an optimum at a vertex of the feasible region.

Equality form

The version used by scipy.optimize.linprog in ar.py is closer to

$$\begin{aligned} \text{minimize} \quad & c^\top x \\ \text{subject to} \quad & A_{\text{eq}} x = b_{\text{eq}}, \\ & x \ge 0. \end{aligned}$$

This is called an *equality-constrained* or *standard-form* linear program. Inequalities can be converted into equalities by adding slack or surplus variables. For example, $2p + q \ge 5$ is equivalent to

$$2p + q - s = 5, \qquad s \ge 0,$$

where s records the amount by which the left-hand side exceeds the minimum requirement.

A farmer example

A farmer can buy two fertilizer mixes:

	nitrogen	phosphorus	cost
P	2	1	3
Q	1	3	2

The field needs at least 5 units of nitrogen and at least 5 units of phosphorus. If $p, q \geq 0$ are the numbers of bags of P and Q, the problem is

$$\begin{aligned} &\text{minimize} && 3p + 2q \\ &\text{subject to} && 2p + q \geq 5, \\ &&& p + 3q \geq 5, \\ &&& p, q \geq 0. \end{aligned}$$

The two nutrient constraints meet at

$$2p + q = 5, \qquad p + 3q = 5,$$

which gives $p = 2$, $q = 1$. The cost is $3 \cdot 2 + 2 \cdot 1 = 8$.

Why is this optimal? The feasible region lies above both lines $2p + q = 5$ and $p + 3q = 5$ in the first quadrant. Moving the objective line $3p + 2q =$ constant downward, the first feasible point it touches is the intersection $(2, 1)$. Other vertices are more expensive: using only P requires $p = 5$, cost 15; using only Q requires $q = 5$, cost 10. Thus the best solution is two bags of P and one bag of Q.

A third mix R containing only potassium and sulfur would receive weight 0 in an optimal solution, because it helps none of the two constraints we care about while increasing the cost. This simple observation is the part that will matter for AR: a linear program can automatically ignore irrelevant ingredients.

Returning to AR

In AR, the "ingredients" are not bags of fertilizer but previously known linear facts. The target is not a nutrient requirement but a new equality whose proof we want to explain. The question becomes:

Can the target equality be obtained as a linear combination of the already registered facts?

This question is exactly an equality-form linear program. The matrix `self.A` stores the registered facts as columns. The vector `b_eq` stores the target equality we want to reproduce. Solving

$$A_{\text{eq}} x = b_{\text{eq}}, \qquad x \geq 0$$

tells AR which facts were actually needed.

There is one technical wrinkle. In ordinary linear algebra one may combine equations with positive or negative coefficients, but `linprog` uses non-negative variables. Therefore `Table.register` stores two columns per fact: one for using the fact forward and one for using it backward. The signed coefficient is the difference between those two non-negative weights.

Back to the bisector example

For the worked example above, the registered facts give the following matrix:

```
        fact1+  fact1-  fact2+  fact2-  fact3+  fact3-
d(ac)   [ 1      -1      -1       1       1      -1   ]
d(cd)   [-2       2       0       0       0       0   ]
d(bc)   [ 1      -1      -1       1      -2       2   ]
pi      [ 0       0       1      -1       1      -1   ]
d(ce)   [ 0       0       2      -2       0       0   ]
d(ab)   [ 0       0       0       0       1      -1   ]
```

Asking *why* $d(CD) - d(CE) = \frac{1}{2}\pi$ produces a target vector with $+1$ on `d(cd)`, -1 on `d(ce)`, and $-\frac{1}{2}$ on `pi`. Solving the equality-form LP gives:

```
x* = (0, 0.5, 0, 0.5, 0, 0)      objective = -1
```

Thus *fact1* and *fact2* are used, each in the backward direction with weight 1/2, while the distractor *fact3* receives weight 0. This is the AR analogue of the farmer's irrelevant mix R: it is available, but it is not needed to produce the target. The proof traceback therefore reports exactly `['fact1', 'fact2']`.

30.6. **A Closer Look at the Language Model.** DD and AR are sound and exhaustive, but they cannot invent a new point. When both get stuck, the LM proposes one - a *rabbit* (Section 1) - which DD+AR then tries again. Two things make this work with a model as small as 150M parameters: how its training data was generated, and how little it is actually asked to do at inference time.

How one can make the training data. The original paper [TWL+24] describes the central idea: avoid human-written proof demonstrations by manufacturing many synthetic geometry problems. For our purposes, the important point is not the exact engineering setup used by DeepMind, but the mathematical recipe it suggests. The whole pipeline can be read as a way of turning solved random diagrams into supervised examples for auxiliary-point prediction.

The recipe is roughly this:

(1) **Generate a random construction.** Start from a small set of free points, then repeatedly apply construction rules such as "take an intersection," "draw a midpoint," "construct an incenter," or "put a point on a line." In code, these are the same kinds of actions listed in `defs.txt`.
(2) **Close the diagram under DD+AR.** Treat the generated construction as a problem state and run the symbolic engine until saturation. The result is a large collection of true consequences: collinearities, equal angles, congruent segments, parallel lines, perpendicular lines, and so on.
(3) **Pick a derived fact as a theorem.** Any nontrivial derived fact can become the goal of a synthetic problem. For instance, if DD+AR derives `perp C D C E`, we may ask a future solver to prove exactly that statement.
(4) **Trace the proof backwards.** Use the proof dependencies to find which earlier facts and which constructed points were actually needed. This is where `trace_back.py` enters: it cuts away irrelevant parts of the random diagram and keeps a smaller proof core.
(5) **Hide one auxiliary point.** Compare the points needed to *state* the theorem with the points needed in the proof. A point used in the proof but absent from the statement is precisely the kind of auxiliary point that human solvers often invent. Remove that construction from the visible problem, and use it as the target output for the language model.

In slogan form: random construction creates a large true diagram; DD+AR discovers many theorems inside it; traceback extracts a small proof; hiding one proof-only point turns the theorem into a rabbit-prediction training example. The paper reports doing this at very large scale: about one billion random diagrams, reduced by proof search and deduplication to roughly 100 million unique examples, of which about 9 million needed at least one auxiliary point.

What we will try to rebuild

For our own basic AlphaGeometry-style system, we do not need the full industrial-scale pipeline. We need a small version with the same logical shape: generate random construction sequences, run DD+AR, select derived goals, trace dependencies, detect proof-only points, and serialize examples of the form "problem state $\mapsto$ useful auxiliary construction." This will give us a local synthetic dataset we understand from end to end.

What we can verify directly. The checkpoint and its config are public, and we confirmed the following ourselves while writing these notes (by independently converting Google's released weights and loading the result): a 12-layer decoder-only transformer, embedding dimension 1024, 8 heads, T5-style relative-position attention, about 152M parameters, and a vocabulary of under 1024 tokens - DSL symbols only, never natural language. Both the input (current stuck state) and output (proposed construction) are the same compact token format seen throughout these notes, e.g. `e : C a c e 02 C b d e 03 ;` for `e = on_line e a c, on_line e b d`. At inference, `lm_inference.py` just runs beam search over this tiny vocabulary and returns a handful of scored candidates - Section 29 showed this loop end to end.

What we will need to choose ourselves. The public materials give enough information to understand the idea, but not every engineering choice needed to recreate the corpus. For our regeneration effort, we must choose: how to sample random constructions, how large each diagram should be, which derived facts count as interesting goals, how aggressively to deduplicate near-identical examples, and how to format the final input-output strings. Those choices are not a weakness of the project; they are exactly where a course version can stay small, transparent, and mathematically inspectable.

30.7. **A Closer Look at `graph.py`.** Every fact and table used so far has to live somewhere between calls to `dd.py` and `ar.py`. That somewhere is a single `Graph` object, built once per problem and threaded through the entire solve: it holds every point, line, and segment seen so far, the `atable`/`rtable`/`dtable` from Section 30.4, and a `cache` dictionary for quick dependency lookups. DD and AR only ever read from and write into this one object.

Two different kinds of "related". The proof graph distinguishes two relationships that are easy to conflate:

- *Adjacency* (`edge_graph`, queried via `neighbors()`): a genuine structural link, e.g. "point P lies on line ℓ." This is the actual geometric picture.
- *Equivalence* (`merge_graph`, queried via `rep()`): a record that two nodes have been *proven equal* - two lines turning out to be the same line, or two lengths turning out to be the same length. This has nothing to do with adjacency.

Equivalence is maintained as a union-find structure, documented directly in `geometry.py`'s `Node` class:

```
  a -> b -
           \
        c -> d -> e -> f -> g

d.merged_to = e
d.rep = g
d.merged_from = {a, b, c, d}
d.equivs = {a, b, c, d, e, f, g}
```

`rep()` just walks the chain of `rep_by` pointers to the representative; two nodes count as "the same" exactly when they share one.

Where $d(\cdot)$ and $l(\cdot)$ actually come from. Section 4.3 and Section 30.4 both used $d(AB)$ as notation for line AB's direction. It is not just notation - `connect_val` builds that exact string:

```python
def connect_val(self, node, deps):
    if node._val:
        return node._val
    name = None
    if isinstance(node, Line):
        name = 'd(' + node.name + ')'
    if isinstance(node, Angle):
        name = 'm(' + node.name + ')'
    if isinstance(node, Segment):
        name = 'l(' + node.name + ')'
    if isinstance(node, Ratio):
        name = 'r(' + node.name + ')'
    v = self.new_node(gm.val_type(node), name)
    self.connect(node, v, deps=deps)
    return v
```

Every `Line`, `Segment`, `Angle`, and `Ratio` node has a separate *value node* - its direction, length, measure, or ratio - reachable as `node._val`. The geometric object and its value are deliberately kept apart, because two different segments can share a length without being the same segment.

Why `cong` merges lengths, not segments. This split is what makes congruence work correctly. Proving $AB \cong CD$ calls:

```python
def make_equal(self, x, y, deps):
    self.connect_val(x, deps=None)
    self.connect_val(y, deps=None)
    vx, vy = x._val, y._val
    self.merge([vx, vy], deps)
```

which merges `x._val` and `y._val` - the two *Length* nodes - not `x` and `y` themselves. Afterwards A, B, C, D are still four distinct points and AB, CD are still two distinct segments, but `rep()` on their Length nodes

now agrees, under a name like `l(ab)`. Checking "are AB and CD congruent?" later is then a single `rep()` comparison, however many other segments have separately been proven equal to either one.

References

[CTO+25] Yuri Chervonyi, Trieu H Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang H Nguyen, Marcelo Menegali, Junehyuk Jung, Junsu Kim, Vikas Verma, Quoc V Le, et al. Gold-medalist performance in solving olympiad geometry with alphageometry2. *Journal of Machine Learning Research*, 26(241):1–39, 2025.

[Har13] Robin Hartshorne. *Geometry: Euclid and beyond*. Springer Science & Business Media, 2013.

[KHS21] Ryan Krueger, Jesse Michael Han, and Daniel Selsam. Automatically building diagrams for olympiad geometry problems. In *CADE*, pages 577–588, 2021.

[TWL+24] Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

Department of Mathematics, Brown University, Providence, RI 02906, USA
Email address: steven_creech@brown.edu

Department of Mathematics, Brown University, Providence, RI 02906, USA
Email address: peter_zenz@brown.edu